Complessità del calcolo

Dipartimento di Elettronica, Informazione e Bioingegneria
Politecnico di Milano

2 aprile 2025

La complessità del calcolo

Quanto efficientemente risolviamo un problema?

- Razionalizzazione dei concetti di:
  - *costo* di una computazione (tempo e memoria)
  - misura e confronto del costo di soluzioni a un problema
- Primo passo: costruire strumenti per valutare la complessità di un calcolo
- Secondo passo: analisi di algoritmi e strutture dati notevoli
  - Alcuni problemi sono molto ricorrenti nella pratica
- Obiettivo: saper progettare e combinare algoritmi e strutture dati per realizzare soluzioni *efficienti* (oltre che corrette)

La complessità del calcolo

Quanto efficientemente risolviamo un problema?

- Posto che un dato problema sia calcolabile, ci domandiamo: a che costo?
- Effettueremo analisi *quantitative* di:
  - Tempo di calcolo impiegato
  - Spazio occupato (registri, caches, RAM, disco, nastro)
- Analisi fatta su criteri di costo oggettivi (e formalizzabili): non terremo conto di:
  - costi di sviluppo, organizzazione del software, testing
  - compromessi dell'ambiente di sviluppo tra obiettivi contrastanti

Oltre la (sola) risolvibilità

Dipendenza dal formalismo di calcolo

- Per la tesi di Church-Turing, un problema è calcolabile o meno indipendentemente dallo strumento usato (purché Turing completo)
- Possiamo dire lo stesso della complessità del calcolo?
  - Una somma in unario ha efficienza diversa da una in base $b > 1$
  - Calcolare una traduzione $y = \tau(x)$ decidendo se $\exists z \in L_\tau = \{x \blacklozenge y | y = \tau(x)\}$ può essere molto meno efficiente del calcolare la traduzione
- È verosimile assumere che cambiando modello di calcolo non cambi il tempo di esecuzione? Intuitivamente, no.

Costruire uno strumento generale

Costo di calcolo "indipendente" dal formalismo

- Non abbiamo una "tesi di Church-Turing" della complessità
- Ci serve uno strumento per valutare la complessità temporale e spaziale:
  - che tralasci "considerazioni superflue"
  - utilizzabile per la maggioranza dei modelli di calcolo
- Non avendo un formalismo di calcolo "preferito" per costruire il modello, partiamo dalle MT *deterministiche*
  - Almeno una MT, per preservare le capacità espressive
  - Una MT deterministica, perchè non possiamo costruire fisicamente una ND

Complessità temporale e spaziale

Complessità temporale

- Data la computazione $c_0 \overset{*}{\vdash} c_r$ di $\mathcal{M}$ la complessità temporale è $T_{\mathcal{M}}(x) = r$ se $\mathcal{M}$ si ferma in un dato c_r, ∞ altrimenti
- $\mathcal{M}$ è *deterministica* $\Rightarrow$ computazione unica sull'ingresso x

Complessità spaziale

- Data la computazione $c_0 \vdash^* c_r$ di $\mathcal{M}$ la complessità spaziale è $S_{\mathcal{M}}(x) = \sum_{j=1}^{k} \max_{i \in \{0,\dots,r\}}(|\alpha_{ij}|)$ con $|\alpha_{ij}|$ lunghezza del contenuto del j-esimo nastro alla mossa i-esima
  - Intuitivamente: la somma delle quantità massime di nastro occupate, per ogni nastro
- NB: $\forall x, \exists k \in \mathbb{N}, \frac{S_{\mathcal{M}}(x)}{k} \leq T_{\mathcal{M}}(x)$: mi serve almeno il tempo per scrivere i simboli

Esempio: riconoscere $L = \{wcw^R\}$ con MT a $k = 1$ nastro

Complessità temporale

- Se $x \in L$, $T_{\mathcal{M}}(x) = |x| + 2$ mosse
- Se $x \notin L$, $x = \alpha a.ucu^R.b\beta$, $T_{\mathcal{M}}(x) = |\alpha a.ucu^R.b|$ mosse
- Se $x \notin L, x \in \{a, b\}^*$, $T_{\mathcal{M}}(x) = |x| + 1$ mosse
- Se $x \in c^+.\{a, b\}^*$, $T_{\mathcal{M}}(x) = 2$ mosse

Complessità spaziale

- Se $x \in L$, $S_{\mathcal{M}}(x) = \left\lfloor \frac{|x|}{2} \right\rfloor$
- Se $x \notin L, x \in \{a, b\}^*$, $S_{\mathcal{M}}(x) = |x| + 1$
- Se $x \in c^+$, $S_{\mathcal{M}}(x) = 0$

Riducendo all'essenziale

Analisi su MT

- Contempla tutti i dettagli del funzionamento della MT
  - La complessità temporale dipende dal valore dell'input
  - Spesso più dettagliata di quanto ci interessi

Una prima semplificazione

- Da $T_{\mathcal{M}}(x)$ e $S_{\mathcal{M}}(x)$ a $T_{\mathcal{M}}(n)$ e $S_{\mathcal{M}}(n)$, dove n è la *lunghezza* dell'input
  - Nel caso precedente $n = |x|$
  - Esempi pratici: num. righe $\times$ num. colonne di una matrice, numero di record in una tabella, numero di pacchetti in arrivo dalla rete
- NB: Si tratta di una semplificazione: in generale abbiamo che $|x_1|{=}|x_2| \not\Rightarrow T_{\mathcal{M}}(x_1){=}T_{\mathcal{M}}(x_2)$, $|x_1|{=}|x_2| \not\Rightarrow S_{\mathcal{M}}(x_1){=}S_{\mathcal{M}}(x_2)$

Gestire la variabilità dell'ingresso

Scelte possibili (sia per $T_{\mathcal{M}}(\cdot)$ che per $S_{\mathcal{M}}(\cdot)$)

- Caso pessimo: $T_{\mathcal{M}}(n) = \max_{x,|x|=n} T_{\mathcal{M}}(x)$
- Caso ottimo: $T_{\mathcal{M}}(n) = \min_{x,|x|=n} T_{\mathcal{M}}(x)$
- Caso medio: $T_{\mathcal{M}}(n) = \sum_{x,|x|=n} \Pr(\mathcal{X} = x) T_{\mathcal{M}}(x)$ (con input $\sim \mathcal{X}$)

Scelte tipiche

- Consideriamo (quasi) sempre il caso pessimo:
  - È quello più rilevante: vogliamo essere sicuri di finire sempre in tempo
  - L'analisi risulta più semplice del caso medio

Tasso di crescita

Quanto crescono $T_{\mathcal{M}}(n)$ e $S_{\mathcal{M}}(n)$?

- Sapere i valori esatti di $T_{\mathcal{M}}(n)$ e $S_{\mathcal{M}}(n)$ per un dato n è utile, ma non fornisce un'idea globale del comportamento dell'algoritmo
  - Dovremmo calcolarli per ogni n di nostro interesse!
- Focalizziamoci su quanto $T_{\mathcal{M}}(n)$ e $S_{\mathcal{M}}(n)$ crescono in funzione di n
- Osserviamo quindi $T_{\mathcal{M}}(n)$ e $S_{\mathcal{M}}(n)$ quando $n \to \infty$
  - Non distingueremo quindi $T_{\mathcal{M}}(n) = n^2 + 13n$ da $3n^2$, hanno comportamenti "simili"
- Semplificazione aggressiva (stiamo dicendo che $n^2 \approx 10^{80} n^2$) ma molto efficace se usata con raziocinio

Comportamento asintotico e notazione

Indicare i tassi di crescita

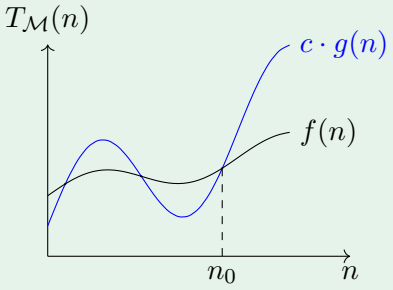
- Introduciamo una notazione per indicare il comportamento asintotico di una funzione:
  - La notazione $\mathcal{O}$-grande: limite asintotico superiore
  - La notazione Ω-grande: limite asintotico inferiore
  - La notazione Θ-grande: limite asintotico sia sup. che inf.
- Caveat pratico 1: comportamento asintotico $\rightarrow$ per valori piccoli di n potrebbe non essere un buon modello
- Caveat pratico 2: in qualche (raro) caso valore "piccolo" per n non corrisponde alla nostra intuizione
  - Un algoritmo con complessità asintotica minore può essere più lento di uno a complessità maggiore per tutti i valori di n che ci interessano
  - Tutto funziona ... fin quando non crescono i dati in ingresso

Notazione $\mathcal{O}$-grande

Definizione

- $\mathcal{O}(g(n))$ è l'insieme di funzioni $f(n)$ con dominio in $\mathbb{N}$
  $\mathcal{O}(g(n)) = \{f(n) \mid \exists c > 0, n_0 > 0$ tali che $\forall n > n_0,\ f(n) \leq c \cdot g(n)\}$

Graficamente



- Le funzioni in $\mathcal{O}(g(n))$ (e.g., $f(n)$) sono dominate da $c \cdot g(n)$ a partire da n_0
- Ogni elemento di $\mathcal{O}(g(n))$ ci dà un limite superiore al valore di $f(n)$ per $n \to \infty$ (a meno di una costante)

Esempi

Facilmente

- $3n^2 + 12n + 35 \in \mathcal{O}(n^2)$
- $5n^3 + 3 \in \mathcal{O}(n^3)$
- $2\log(n) + \log(\log(n)) \in \mathcal{O}(\log(n))$

Leggermente meno facilmente

- $3n^2 + 12n + 35 \in \mathcal{O}(n^{20})$
- $5n^3 + 3 \in \mathcal{O}(2^n)$
- $2\log(n) + \log(\log(n)) \in \mathcal{O}(n)$

Un abuso notazionale comunemente accettato

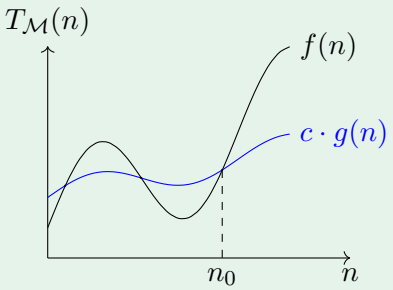
- È comune usare $f(n) = \mathcal{O}(g(n))$ al posto di $f(n) \in \mathcal{O}(g(n))$ (o $f(n)$ è $\mathcal{O}(g(n))$)

Notazione Ω-grande

Definizione (Knuth)

- $\Omega(g(n))$ è l'insieme di funzioni $f(n)$ con dominio in $\mathbb{N}$
  $\Omega(g(n)) = \{g(n) \mid \exists c > 0, n_0 > 0$ tali che $\forall n > n_0,\ f(n) \geq c \cdot g(n)\}$

Graficamente



- Le funzioni in $\Omega(g(n))$ (e.g., $f(n)$) dominano $c \cdot g(n)$ a partire da n_0

Esempi

Direttamente dalla definizione

- $3n^2 + 12n + 35 \in \Omega(n^2)$
- $7n^2 \log(n) + 15 \in \Omega(n^2 \log(n))$
- $n2^n + n^{50} \in \Omega(n2^n)$

Di conseguenza, anche

- $3n^2 + 12n + 35 \in \Omega(n)$
- $5n^4 + 3 \in \Omega(log(n))$, ma $5n^4 + 3 \notin \Omega(n^4 log(n))$
- $n2^n + n^{13} \in \Omega(2^n)$, ma $n2^n + n^{13} \notin \Omega(2^{n^2})$

Proprietà

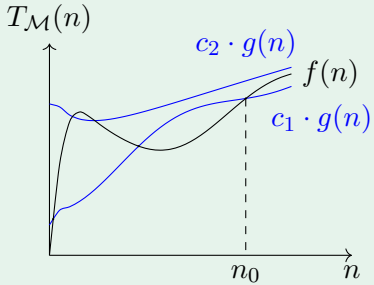
- Se e solo se $f(n) \in \mathcal{O}(g(n))$ allora $g(n) \in \Omega(f(n))$

Notazione Θ-grande

Definizione

- $\Theta(g(n))$ è l'insieme di funzioni $f(n)$ con dominio in $\mathbb{N}$
  $\Theta(g(n)) = \{g(n) \mid \exists c_1 > 0, c_2 > 0, n_0 > 0$ tali che
  $\forall n > n_0,\ c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)\}$

Graficamente



- Le funzioni in $\Theta(g(n))$ sono comprese tra $c_1 \cdot g(n)$ e $c_2 \cdot g(n)$ a partire da n_0

Esempi

Direttamente dalla definizione

- $3n^2 + 12n + 35 \in \Theta(n^2)$
- $7n^2 \log(n) + 15 \in \Theta(n^2 \log(n))$
- $n2^n + n^{50} \in \Theta(n2^n)$

Tuttavia

- $3n^2 + 12n + 35 \notin \Theta(n)$
- $3n^2 + 12n + 35 \notin \Theta(\log(n))$
- $n2^n + n^{50} \notin \Theta(2^n)$
- $n2^n + n^{50} \notin \Theta(n^{100})$

Proprietà di $\mathcal{O}, \Omega, \Theta$

Proprietà di Θ

- appartenere a $\Theta(f(n))$ è una relazione di equivalenza sull'insieme di funzioni
  - Riflessiva: $f(n) \in \Theta(f(n))$
  - Simmetrica: $f(n) \in \Theta(g(n)) \Leftrightarrow g(n) \in \Theta(f(n))$
  - Transitiva: $f(n) \in \Theta(g(n)) \ \wedge \ h(n) \in \Theta(g(n)) \ \Rightarrow \ f(n) \in \Theta(h(n))$

Proprietà di $\mathcal{O}, \Omega$

- Esse sono riflessive: $f(n) \in \mathcal{O}(f(n))$, $f(n) \in \Omega(f(n))$
- $f(n) \in \Theta(g(n))$ se e solo se $f(n) \in \mathcal{O}(g(n)) \wedge f(n) \in \Omega(g(n))$
- $\mathcal{O}$ e Ω sono relazioni d'ordine parziale

Confronti operativi

Definizioni come limiti

- Se $\lim_{n\to\infty} \frac{f(n)}{g(n)} = c, c \neq 0$ e $c \neq \infty$ allora $f(n) \in \Theta(g(n))$
  - Gli andamenti asintotici di f e g differiscono per una costante moltiplicativa
- Se $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$ allora $f(n) \in \mathcal{O}(g(n))$, ma $f(n) \notin \Theta(g(n))$
  - Il comportamento di $f(n)$ è diverso in modo sostanziale da quello di $g(n)$ (in particolare, cresce più velocemente)
  - Si indica anche, con notazione più sintetica, $\Theta(f(n)) < \Theta(g(n))$
- L'uso della notazione $\mathcal{O}, \Omega, \Theta$ estrae la porzione "più importante" di una funzione di complessità: il comportamento a meno di una costante moltiplicativa

Una prima applicazione

Riconoscere $L = \{wcw^r\}$ con MT a 1 nastro

- $T_{\mathcal{M}}(n)$ è $\Theta(n)$, $S_{\mathcal{M}}(n)$ è anch'essa $\Theta(n)$
- Ci sono possibilità di miglioramento (= soluzioni a complessità inferiore)?
- Per $T_{\mathcal{M}}(n)$, è ben difficile (devo leggere tutta la stringa)
- Per $S_{\mathcal{M}}(n)$, sì. Ad esempio, con MT a 2 nastri:
  - Memorizzo solo la posizione del carattere da confrontare (in binario)
  - Sposto la testina e confronto i caratteri in posizione i e $n - i + 1$
- NB: la complessità spaziale tiene conto solamente le celle dei nastri memoria
  - Unica eccezione: MT a nastro singolo, dove contiamo le celle dell'unico nastro

Pseudocodice per MT a 2 nastri $\mathtt{N_a}, \mathtt{N_b}$

1. Ciclo per trovare c, ad ogni passo incrementa il contatore memorizzato $\mathtt{N_a}$ [$T_{\mathcal{M}}(n)= n\log(n)$, $S_{\mathcal{M}}(n)=\log(n)$]
2. Ripeti fino a quando il contatore su $\mathtt{N_a}$ è $= 0$
    1. Copia il contenuto di $\mathtt{N_a}$ su $\mathtt{N_b}$ [$T_{\mathcal{M}}(n)=S_{\mathcal{M}}(n)=\log(n)$]
    2. Sposta la testina di ingresso di $\mathtt{N_b}$ posti a sx, decrementando $\mathtt{N_b}$ [$T_{\mathcal{M}}(n)= i\log(i)$, $S_{\mathcal{M}}(n)=\log(n)$]
    3. Memorizza il carattere nello stato [$T_{\mathcal{M}}(n)=S_{\mathcal{M}}(n)=k$]
    4. Ritorna al carattere c [$T_{\mathcal{M}}(n)=i$,$S_{\mathcal{M}}(n)=k$]
    5. Copia il contenuto di $\mathtt{N_a}$ su $\mathtt{N_b}$ [$T_{\mathcal{M}}(n)=S_{\mathcal{M}}(n)=\log(n)$]
    6. Sposta la testina di ingresso di $\mathtt{N_b}$ posti a dx, decrementando $\mathtt{N_b}$ [$T_{\mathcal{M}}(n)= i\log(i)$, $S_{\mathcal{M}}(n)=\log(n)$]
    7. Confronta il carattere con lo stato, se diversi halt [$T_{\mathcal{M}}(n)=S_{\mathcal{M}}(n)=k$]
    8. Decrementa $\mathtt{N_a}$ e torna a c [$T_{\mathcal{M}}(n)= \log(i)+i$, $S_{\mathcal{M}}(n)=\log(n)$]

Considerazioni sul $T_{\mathcal{M}}(n)$ e $S_{\mathcal{M}}(n)$

Compromessi tra spazio e tempo

- La nuova soluzione è $S_{\mathcal{M}}(n) \in \Theta(\log(n))$, ma $T_{\mathcal{M}}(n) \in \Theta(n^2 \log(n))$
- È un caso di compromesso spazio-temporale: la soluzione è più lenta ma occupa meno spazio
  - È possibile spesso precalcolare alcuni risultati utilizzati spesso e andare a rileggerli da memoria al posto di ricalcolarli da zero
- Questo esempio giustifica perchè in una MT a k nastri, per convenzione la testina di ingresso si muove in entrambe le direzioni
  - Farla muovere in una sola direzione non cambia il potere computazionale della MT ...
  - ... ma previene esempi (non banali) di algoritmi con $S_{\mathcal{M}}(n)$ sub-lineare

Modelli di calcolo

Altri modelli *deterministici* di calcolo

- FSA: hanno sempre $S_{\mathcal{FSA}}(n) \in \Theta(1)$, $T_{\mathcal{FSA}}(n) \in \Theta(n)$
  - Tecnicamente, $T_{\mathcal{FSA}}(n) = n$, leggono un carattere per mossa
- APD: hanno sempre $S_{\mathcal{APD}}(n) \in \mathcal{O}(n)$, $T_{\mathcal{APD}}(n) \in \Theta(n)$
- MT a nastro singolo?
  - È facile trovare una soluzione $T_{\mathcal{M}}(n) \in \Theta(n^2)$ per il riconoscimento di $L = \{wcw^r\}$
  - $S_{\mathcal{M}}(n)$ non potrà mai essere minore di $\Theta(n)$
  - Non esiste un algoritmo più efficiente di $T_{\mathcal{M}}(n) \in \Theta(n^2)$ per MT a nastro singolo (Intuizione: a ogni controllo di ognuna delle $\frac{n-1}{2}$ coppie di lettere, la TM scandisce una porzione di nastro che passa su c)
- In generale: MT a nastro singolo sono più potenti degli APD, ma ciò che eseguono può avere qualunque complessità spazio/temporale!

I teoremi di accelerazione lineare

Teorema (Compressione dello spazio)

Se L è accettato da una MT $\mathcal{M}$ a k nastri in $S_{\mathcal{M}}(n)$, per ogni $c \in \mathbb{Q}, c > 0$ posso costruire una MT $\mathcal{M}'$ a k nastri con $S_{\mathcal{M}'}(n) < c \cdot S_{\mathcal{M}}(n)$

Schema della dimostrazione

- Scelgo un fattore di compressione intero r tale che $r \geq c$
- Per ognuno degli i nastri di $\mathcal{M}$, considero l'alfabeto Γ_i e costruisco Γ_i' di $\mathcal{M}'$ creando un elemento in Γ_i' per ogni $s \in \Gamma_i^r$
- Costruisco l'OC di $\mathcal{M}'$ in modo tale per cui:
  - Calcoli con i nuovi simboli sui nastri emulando le mosse di $\mathcal{M}$ spostando le testine sui nastri di $\mathcal{M}'$ ogni r movimenti di $\mathcal{M}$
  - Memorizzi la posizione della testina "all'interno" dei nuovi simboli degli alfabeti di nastro Γ_i usando gli stati

I teoremi di accelerazione lineare - 2

Teorema (Da k a 1 nastro di memoria)

Se L è accettato da una MT $\mathcal{M}$ a k nastri in $S_{\mathcal{M}}(n)$, posso costruire una MT $\mathcal{M}'$ a 1 nastro (non nastro singolo) con $S_{\mathcal{M}'}(n) = S_{\mathcal{M}}(n)$ (concateno i contenuti dei k nastri su uno solo)

Teorema

Se L è accettato da una MT $\mathcal{M}$ a k nastri in $S_{\mathcal{M}}(n)$, per ogni $c \in \mathbb{Q}, c > 0$ posso costruire una MT $\mathcal{M}'$ a $k = 1$ nastri con $S_{\mathcal{M}'}(n) < c \cdot S_{\mathcal{M}}(n)$

I teoremi di accelerazione lineare - 3

Teorema

Se L è accettato da una MT $\mathcal{M}$ a k nastri in $T_{\mathcal{M}}(n)$, per ogni $c \in \mathbb{Q}, c > 0$ posso costruire una MT $\mathcal{M}'$ a $k+1$ nastri con $T_{\mathcal{M}'}(n) = \max(n+1, c \cdot T_{\mathcal{M}}(n))$

Schema di dimostrazione

- Approccio simile alla complessità spaziale: codifichiamo in modo compresso i simboli dell'alfabeto di $\mathcal{M}$ e calcoliamo sui simboli compressi
- Dobbiamo considerare che la compressione è fatta a runtime: il minimo costo per effettuare la compressione è lineare nel numero di simboli da comprimere
- Comprimendo r simboli in uno, nel caso pessimo, possono servirmi 3 mosse di $\mathcal{M}'$ per emularne $r+1$ di $\mathcal{M}$

Conseguenze pratiche

L'unico "pasto gratis" è lo speedup lineare

- Lo schema di dimostrazione usato vale anche per i calcolatori a-la Von Neumann
  - Equivale a cambiare la dimensione della word della macchina ($32\text{b} \rightarrow 64\text{b}$) o, equivalentemente, ad usare operazioni vettoriali (SSE/AVX,Neon,Altivec)
- Possiamo avere speedup *lineari* arbitrariamente grandi, aumentando il parallelisimo fisico (stanti i limiti della termodinamica/trasmissione dei segnali)
- Miglioramenti più che lineari nel tempo di calcolo possono essere ottenuti solo *cambiando algoritmo*
  - Concepire/utilizzare algoritmi efficienti è di gran lunga più efficace della forza bruta

Modelli di calcolo a confronto

MT vs. calcolatori reali

- Differenze "marginali": un calcolatore è in grado di fare operazioni aritmetiche su tipi a dimensione finita in tempo costante (e.g., add in un ciclo di clock), la MT necessita di propagare gli effetti al singolo bit uno per uno
  - Il calcolatore opera con un alfabeto molto vasto, $|\mathbf{I}| = 2^w$, dove w è la dimensione della parola architetturale
- Un calcolatore può accedere direttamente ad una cella di memoria, una MT impiega $\Theta(n)$ dove n è la distanza della stessa dalla posizione della testina
- Cambiamo modello di calcolo, avvicinandoci ai calcolatori reali

La macchina RAM

Un calcolatore semplificato

- La macchina RAM ha di un nastro di lettura `In` e uno di scrittura `Out` come la MT
- Assumiamo il programma cablato nell'OC, così come la logica del program counter
- La RAM è dotata di una memoria con accesso a indirizzamento diretto $\mathtt{M}[n], n \in \mathbb{N}$ al posto dei nastri di memoria: l'accesso non necessita di scorrimento delle celle
- Le istruzioni di un programma usano come primo operando sorgente e come operando destinazione $\mathtt{M}[0]$
- Ogni cella contiene *un intero* ($x \in \mathbb{N}$)

La macchina RAM

Instruction set e semantica pseudo-RTL

Istruzione	Semantica
`LOAD X`	$\texttt{M}[0] \leftarrow \texttt{M}[X]$
`LOAD= X`	$\texttt{M}[0] \leftarrow X$
`LOAD* X`	$\texttt{M}[0] \leftarrow \texttt{M}[\texttt{M}[X]]$
`STORE X`	$\texttt{M}[X] \leftarrow \texttt{M}[0]$
`STORE* X`	$\texttt{M}[\texttt{M}[X]] \leftarrow \texttt{M}[0]$
`ADD X`	$\texttt{M}[0] \leftarrow \texttt{M}[0] + \texttt{M}[X]$
`SUB X`	$\texttt{M}[0] \leftarrow \texttt{M}[0] - \texttt{M}[X]$
`MUL X`	$\texttt{M}[0] \leftarrow \texttt{M}[0] \times \texttt{M}[X]$
`DIV X`	$\texttt{M}[0] \leftarrow \texttt{M}[0]/\texttt{M}[X]$
`HALT`	–

Istruzione	Semantica
`READ X`	$\texttt{M}[X] \leftarrow \texttt{In}$
`READ* X`	$\texttt{M}[\texttt{M}[X]] \leftarrow \texttt{In}$
`WRITE X`	$\texttt{Out} \leftarrow \texttt{M}[X]$
`WRITE= X`	$\texttt{Out} \leftarrow X$
`WRITE* X`	$\texttt{Out} \leftarrow \texttt{M}[\texttt{M}[X]]$
`JUMP l`	$\texttt{PC} \leftarrow l$
`JZ l`	Se $\texttt{M}[0] = 0$ $\texttt{PC} \leftarrow l$

Ci sono `JGT`, `JGZ`, `JLT`, `JLZ`

Test (semplice) di primalità in assembly RAM

Stampa 1 se l'intero su nastro è primo 0 altrimenti

```
 1          READ 1
 2          LOAD= 1
 3          SUB 1
 4          JZ no
 5          LOAD= 2
 6          STORE 2
 7  loop:   LOAD 1
 8          SUB 2
 9          JZ yes
10          LOAD 1
11          DIV 2
12          MUL 2
13          SUB 1
14          JZ no
15          LOAD 2
16          ADD= 1
17          STORE 2
18          JUMP loop
19  yes:    WRITE= 1
20          HALT
21   no:    WRITE= 0
22          HALT
```

Complessità del precedente programma

Analisi di complessità temporale e spaziale

- Per la RAM, valutiamo la complessità in funzione del numero di mosse/numero di celle di memoria
- Assunzione: istruzioni singole a costo costante c_i, dove i è l'indice della riga
- Le istruzioni 1 – 6 sono eseguite al più una volta $\rightarrow$ costo $= c_1 + c_2 + c_3 + c_4 + c_5 + c_6$, è una costante k_1
- Analogamente per le istruzioni 19 – 22, costo costante k_2
- Le istruzioni 7 – 18 hanno costo costante k_3, ma sono eseguite, nel caso pessimo n volte
- $T_{\mathcal{RAM}}(n) = k_1 + nk_3 + k_2 = \Theta(n)$
- $S_{\mathcal{RAM}}(n) = 3 = \Theta(1)$ (uso solo 3 celle di mem)

Analisi di altri algoritmi

Riconoscere $L = \{wcw^R\}$

- $T_{\mathcal{RAM}}(n) = \Theta(n)$
- $S_{\mathcal{RAM}}(n) = \Theta(n)$

Ricerca Binaria

- Input: una sequenza ordinata di interi, ed un numero da cercare in essa
- Output: 1 se l'elemento cercato esiste nella sequenza, 0 altrimenti
- Analizziamo la porzione di codice dopo che:
  - La sequenza è stata caricata in memoria
  - $\mathtt{M}[1]$ e $\mathtt{M}[2]$ contengono gli indirizzi della prima e dell'ultima cella della sequenza stessa

Ricerca Binaria: codice

```
 1          READ 3
 2          LOAD 1
 3          STORE 4
 4          ADD 2
 5          SUB= 1
 6          STORE 5
 7  loop:   LOAD 5
 8          SUB 4
 9          JZ no
10          LOAD 5
11          ADD 4
12          DIV= 2
13          STORE 6
14          LOAD* 6
15          SUB 3
16          JZ yes
17          JLT low
18          JUMP high
19    low:  LOAD 6
20          SUB= 1
21          STORE 5
22          JUMP loop
23   high:  LOAD 6
24          ADD= 1
25          STORE 4
26          JUMP loop
27    yes:  WRITE= 1
28          HALT
29     no:  WRITE= 0
30          HALT
```

Costo complessivo: $T_{\mathcal{RAM}}(n) = \Theta(\log(\texttt{lunghezza_vettore}))$

Limiti del criterio di costo

Numeri molto grandi

- Consideriamo il caso del calcolo di 2^{2^n} con una RAM
- Uno schema di implementazione possibile è

```
read(n); x=2;
for(int i=0 ; i< n; i++) {x = x*x;}
write(x);
```

- Che complessità temporale ha l'implementazione qui sopra?
  $T_{\mathcal{RAM}}(n) = k_1 + nk_2 + k_3 = \Theta(n)$
- Qualcosa non va: mi servono 2^n bit solo per scrivere il risultato!

Un criterio di costo più preciso

Quando contare i singoli bit serve

- Il criterio di costo precedente considera un intero arbitrario di dimensione costante
  - È una semplificazione: "1" è più corto di "1000000"
- L'approssimazione regge fin quando una singola parola della macchina reale contiene gli interi che maneggiamo
- Se questo non accade, dobbiamo tenere conto del numero di cifre necessarie per rappresentare un intero
  - Caricare e salvare interi non è più a costo costante
  - Le operazioni elementari (somma, prodotto...) neppure
  - Anche gli indirizzi di memoria sono interi $\rightarrow$ stesse problematiche nel (raro) caso in cui parola della macchina non riesce a contenere il massimo indirizzo (μC a 8 bit)

Criterio di costo logaritmico

Contare i singoli bit

- Copiare/spostare/scrivere/leggere un intero i costa tanto quanto il suo numero di cifre in base b: $\log_b(i) = \Theta(\log(i))$
  - Con $b = 2$ il costo è il numero di bit usati per rappresentare i
- Il costo delle operazioni aritmetico/logiche elementari dipende dall'operazione (definiamo $d = \log_2(i)$)
  - Addizioni, sottrazioni, op. al bit $\Theta(d)$
  - Moltiplicazioni: metodo scolastico $\Theta(d^2)$
    - Primo miglioramento: $\Theta(d^{\log_2(3)}) \approx \Theta(d^{1.58})$
    - Ulteriore miglioramento: $\Theta(d \log(d) \log(\log(d))$
    - Miglior algoritmo attuale: $\Theta(d \log(d))$
  - Divisioni: metodo scolastico $\Theta(d^2)$, o al costo dell'algoritmo di moltiplicazione scelto per $\log^2(d)$
- `JUMP` e `HALT` sono a costo costante, così come le `Jcc` (il valore del codice di condizione è nella PSW, già calcolato)

Calcolo di 2^{2^n}, complessità a costo logaritmico

Analisi di caso pessimo, MUL scolastica

1		READ 2	$\log(n)$
2		LOAD= 2	$\log(2) = k$
3		STORE 1	$\log(2) = k$
4	loop:	LOAD 1	$\log(2^{2^{n-1}}) = 2^{n-1}$
5		MUL 1	$(\log(2^{2^{n-1}}))^2 = (2^{n-1})^2 = 2^{2n-2}$
6		STORE 1	$\log(2^{2^n}) = 2^n$
7		LOAD 2	$\log(n)$
8		SUB= 1	$\log(n)$
9		STORE 2	$\log(n-1)$
10		JGT loop	1
11		WRITE 1	$\log(2^{2^n}) = 2^n$
12		HALT	1

$T_{\mathcal{RAM}}(n)=\log(n) + n(2^{n-1} + 2^{2n-2} + 2^n + 3\log_2(n)) + 2^n = \Theta(n2^{2n-2})$

Rapporti tra criteri di costo

Ri-analizzando

- Riconoscere $L = \{wcw^R\}$ è $\Theta(n \log(n))$ (colpa del contatore)
- Ricerca binaria: $\Theta(\log(n)^2)$ (colpa degli indici)

Quale criterio scegliere?

- Se la dimensione di ogni singolo elemento in ingresso non varia significativamente nell'esecuzione dell'algoritmo (= stesso numero di cifre per tutta l'esecuzione): costo costante
- Nel caso in cui ci sia un significativo cambio nella dimensione dei singoli elementi in ingresso (= il numero di cifre cresce in modo significativo): costo logaritmico

Rapporti tra complessità con diversi modelli di calcolo

Modelli di calcolo diversi $\rightarrow$ diversa efficienza

- Risolvere lo stesso problema con macchine diverse può dare luogo a complessità diverse
- Non esiste un modello migliore in assoluto

Tesi di correlazione polinomiale

- Sotto "ragionevoli" ipotesi di criteri di costo, se un problema è risolvibile da $\mathcal{M}$ in $T_{\mathcal{M}}(n)$, allora è risolvibile da un qualsiasi altro modello (Turing-completo) $\mathcal{M}'$ in $T_{\mathcal{M}'}(n) = \pi(T_{\mathcal{M}}(n))$ dove $\pi(\cdot)$ è un opportuno polinomio
- Dimostriamo il teorema di correlazione (temporale) polinomiale tra MT e RAM

Correlazione temporale tra MT a k nastri e RAM

RAM simula MT a k nastri: simulazione delle azioni

- Mappiamo la MT sulla RAM:
  - Stato della MT $\rightarrow$ Prima cella di memoria della RAM
  - Una cella RAM per ogni cella del nastro
  - Suddividiamo la restante memoria della RAM in blocchi da k celle
- Riempiamo i blocchi con questa strategia:
  - Blocco 0 : posizione delle k testine
  - Blocco n, $n > 0$: n-esimo simbolo di ognuno dei k nastri
- La RAM emula la lettura di un carattere sotto la testina con un accesso indiretto, usando l'indice contenuto nel blocco 0

Correlazione temporale tra MT a k nastri e RAM

RAM simula MT a k nastri: lettura

- Lettura del il blocco 0 e dello stato ($\Theta(k)$ mosse)
- Lettura dei valori sui nastri in corrispondenza delle testine ($\Theta(k)$ accessi indiretti)

RAM simula MT a k nastri: Scrittura

- Scrittura dello stato ($\Theta(1)$)
- Scrittura delle celle dei nastri ($\Theta(k)$ accessi indiretti)
- Scrittura nel blocco 0 per aggiornare le posizioni delle k testine ($\Theta(k)$))

Complessità dell' emulazione

- RAM emula una mossa della MT con un k di mosse: $T_{\mathcal{RAM}}(n) = \Theta(T_{\mathcal{M}}(n))$ ($= \Theta(T_{\mathcal{M}}(n)\log(T_{\mathcal{M}}(n)))$ costo log.)

Correlazione temporale tra MT a k nastri e RAM

MT a k nastri simula RAM (senza MUL/DIV per semplicità)

- Organizziamo un nastro della MT così:



- Il nastro è inizialmente vuoto: salviamo solo le celle in cui è avvenuta una STORE
- Usiamo un ulteriore nastro per contenere $\mathtt{M}[0]$ in binario
- Usiamo un ultimo nastro come stoccaggio temporaneo per quando serve salvare per la prima volta $\mathtt{M}[i_j]$ ma $\mathtt{M}[i_k]$ e $\mathtt{M}[i_l]$, $i_k < i_j < i_l$, sono state già salvate

Correlazione temporale tra MT a k nastri e RAM

MT a k nastri simula RAM: simulare istruzioni

- `LOAD x`: cerco x sul nastro principale, copio la porzione dopo ♦ nella zona dati di $\texttt{M}[0]$ usando il nastro di supporto
- `STORE x`: cerco x sul nastro principale:
  - Se lo trovo, salvo il valore di $\texttt{M}[0]$
  - Altrimenti, creo dello spazio usando il nastro di servizio se necessario e salvo
- `ADD x`: cerco x, copio $\texttt{M}[x]$ sul nastro di supporto, calcolo la somma scrivendo direttamente in $\texttt{M}[0]$
- In generale: simulare una mossa della RAM richiede alla MT un numero di mosse minore o uguale a $c \times$ (lunghezza del nastro principale)

Correlazione temporale tra MT a k nastri e RAM

Lemma (Occupazione sul nastro principale)

Lo spazio occupato sul nastro principale è $\mathcal{O}(T_{\mathcal{RAM}}(n))$

Dimostrazione.

- Ogni cella della RAM occupa $\log(i_j) + \log(\texttt{M}[i_j])$
- Ogni cella della RAM viene materializzata solo se la RAM effettua una `STORE`
- La `STORE` costa alla RAM $\log(i_j) + \log(\texttt{M}[i_j])$
- Per riempire r celle la RAM ci mette $\sum_{j=1}^{r} \log(i_j) + \log(\texttt{M}[i_j])$ in tempo: quantità identica allo spazio che occupano sul nastro della MT ■

Correlazione temporale tra MT a k nastri e RAM

Concludendo

- La MT impiega al più $\Theta(T_{\mathcal{RAM}}(n))$ per simulare una mossa della RAM
- Se la RAM ha complessità $T_{\mathcal{RAM}}(n)$ essa effettua al più $T_{\mathcal{RAM}}(n)$ mosse (ogni mossa costa almeno 1)
- La simulazione completa della RAM da parte della MT costa al più $\Theta((T_{\mathcal{RAM}}(n))^2)$ il legame tra $T_{\mathcal{RAM}}(n)$ e $T_{\mathcal{MT}}(n)$ tra è polinomiale

Conseguenze della correlazione temporale tra MT RAM

Implicazioni della correlazione polinomiale

- La relazione polinomiale tra il tempo di calcolo su $T_{\mathcal{RAM}}(n)$ e $T_{\mathcal{MT}}(n)$ ci consente di definire la classe P di problemi risolvibili in tempo/spazio polinomiale
- Questo risultato ha portato alla formulazione di una "tesi di trattabilità": i problemi risolvibili in P sono quelli trattabili
- La classe P comprende anche polinomi come n^{30}, ma l'esperienza pratica conferma che la maggioranza dei problemi polinomiali di interesse pratico ha anche un grado del polinomio accettabile

Complessità degli algoritmi

Dipartimento di Elettronica, Informazione e Bioingegneria
Politecnico di Milano

24 aprile 2024

Complessità di un algoritmo

Quanto efficientemente risolviamo un problema?

- Dato un problema, un buon flusso di lavoro è:
  1. Concepiamo un algoritmo che lo risolve
  2. Ne valutiamo la complessità
  3. Se la complessità è soddisfacente, lo implementiamo
- Per la correttezza, non c'è una soluzione in generale
  - ... ma questo non nega a priori la possibilità di dimostrarla per dati casi particolari
- Per valutare la complessità ci serve rappresentare l'algoritmo in una qualche forma

Scelta del linguaggio

Pseudocodice

- Semplice linguaggio di programmazione imperativo
- Tralascia gli aspetti non fondamentali per le nostre analisi
- Facilmente traducibile in C/Java/Python/C++
- Sintassi piuttosto asciutta (simile a Python)
- È possibile effettuare analisi di complessità anche su codice scritto in un qualunque linguaggio di programmazione
  - La tecnica resta la stessa dello pseudocodice

Pseudocodice - Sintassi

Procedure, assegnamenti, costrutti di controllo

- Ogni algoritmo è rappresentato con una procedura (= funzione che modifica i dati in input, non ritorna nulla)
- Operatori: Aritmetica a singola precisione come in `C`, assegnamento ($\leftarrow$), e confronti ($<, \leq, =, \geq, >, \neq$)
- Commenti mono-riga con $\triangleright$, blocchi dati dall'indentazione
- Costrutti di controllo disponibili: `while`, `for`, `if-else`
- Tutte le variabili sono locali alla procedura descritta
- Il tipo delle variabili non è esplicito, va inferito dal loro uso

Pseudocodice

Tipi di dato aggregato

- Ci sono gli array, notazione identica al `C`, indici iniziano da 1
- Sono disponibili anche i sotto-array (slices) come in Fortran, Matlab, Python
  - `A[i..j]` è la porzione di array che inizia dall'`i`-esimo elemento e termina al `j`-esimo
- Sono presenti aggregati eterogenei (= strutture `C`)
  - L'accesso a un campo è effettuato tramite l'operatore `.` `A.campo1` è il campo di nome `campo1` della struttura A
  - Diversamente dal C, una variabile di tipo aggregato è un *puntatore* alla struttura
  - Un puntatore non riferito ad alcuna struttura ha valore NIL

Attenzione all'aliasing

1 $y \leftarrow x$
2 $x.f \leftarrow 3$ // dopo questa riga anche $y.f$ vale 3

Pseudocodice - Convenzioni

Passaggio parametri

- Il passaggio di parametri ad una procedura viene effettuato:
  - Nel caso di tipi non aggregati: per copia
  - Nel caso di tipi aggregati: per riferimento
- Comportamento identico al C per tipi non aggregati ed array
- Diverso per le strutture (in C sono passate per copia, uguale a quello di Java)

Modello di esecuzione

- Lo pseudocodice è eseguito dalla macchina RAM
- Assunzione fondamentale: un singolo statement di assegnamento tra tipi base è tradotto in un numero costante k di istruzioni dell'assembly RAM

Criteri per l'analisi

Criterio di costo

- Adottiamo il criterio di *costo costante* per l'esecuzione dei nostri algoritmi
  - La maggioranza degli algoritmi che vedremo non ha espansioni significative della dimensione dei singoli dati
  - Se c'è grande espansione consideriamo dati a precisione multipla come vettori di cifre
- Ogni statement semplice di pseudocodice è eseguito in $\Theta(k)$
- Focalizzeremo la nostra analisi sulla complessità *temporale* degli algoritmi
  - È quella che presenta variazioni più "interessanti" a seconda del tipo di soluzione

Una prima analisi

Cancellare un elemento da una collezione di n elementi

- Salvata in un vettore

CancellaElVett(v, len, e)

```
1  i ← 1
2  while v[i] ≠ e and i < len
3        i ← i + 1
4  while i < len − 1
5        v[i] ← v[i + 1]
6        i ← i + 1
7  if i = len
8        v[i] ← ⊥
```

- Salvata in una lista

CancellaElLista(l, e)

```
1  p ← l
2  if p ≠ NIL and p.value = e
3        l ← l.next
4        return
5  while p.next ≠ NIL and
6           p.next.value ≠ e
7        p ← p.next
8  if p.next.value = e
9        p.next ← p.next.next
```

- Sono entrambi $\Theta(n)$ nel caso pessimo

Un altro esempio

Moltiplicazione di matrici: $dim(A) = \langle n, m \rangle$ $dim(B) = \langle m, o \rangle$

MatrixMultiply(A, B)

```
1  for i ← 1 to n
2      for j ← 1 to o
3          C[i][j] ← 0
4          for k ← 1 to m
5              C[i][j] ← C[i][j] + A[i][k] · B[k][j]
6  return C
```

- La riga 3 viene eseguita $n \cdot o$ volte, la riga 5 viene eseguita $n \cdot m \cdot o$ volte $\rightarrow \Theta(n \cdot m \cdot o)$ (sia nel caso pessimo, che in generale)
- Diventa $\Theta(n^3)$ se le matrici sono quadrate

Ricorsione e complessità

Come calcolare la complessità di algoritmi ricorsivi?

- Ci sono algoritmi con complessità non immediatamente esprimibile in forma chiusa
- Il caso tipico sono algoritmi *divide et impera*:
  - Divido il problema in a sottoproblemi con dimensione dell'input pari a una frazione $\frac{1}{b}$ dell'originale, n
  - Quando n è piccolo a sufficienza, risolvo in tempo costante (caso limite $n = 0$)
  - Ricombino le soluzioni dei sottoproblemi
  - Indichiamo con $D(n)$ il costo del suddividere il problema e con $C(n)$ il costo di combinare le soluzioni
- Esprimiamo il costo totale $T(n)$ con un'*equazione di ricorrenza* (o *ricorrenza*):

$$T(n) = \begin{cases} \Theta(1) \text{ se } n < c \\ D(n) + aT(\frac{n}{b}) + C(n) \text{ altrimenti} \end{cases}$$

Ricorsione e complessità

Come risolvere le ricorrenze?

- Sono possibili 3 tecniche principali:
  - Sostituzione
  - Esame dell'albero di ricorsione
  - Teorema dell'esperto (master theorem)
- Usiamo come caso di studio la ricerca binaria:
  - Formuliamo il problema di cercare in un vettore lungo n come quello di cercare nelle sue metà superiori e inferiori
  - Costo di suddivisione (calcolo indici) costante $D(n) = \Theta(1)$
  - Costo di ricombinazione costante: sappiamo che una delle due metà non contiene per certo l'elemento cercato $C(n) = \Theta(1)$
  - Complessità espressa come $T(n) = \Theta(1) + T(\frac{n}{2}) + \Theta(1)$

Metodo di sostituzione

Ipotesi e dimostrazione

- Il metodo di sostituzione si articola in tre fasi:
  1. Intuire una possibile soluzione
  2. Sostituire la presunta soluzione nella ricorrenza
  3. Dimostrare per induzione che la presunta soluzione è tale per l' equazione/disequazione alle ricorrenze
- Ad esempio, con la complessità della ricerca binaria: $T(n) = \Theta(1) + T(\frac{n}{2}) + \Theta(1)$
  1. Intuizione: penso sia $T(n) = \mathcal{O}(\log(n))$ ovvero $T(n) \leq c\log(n)$
  2. Devo dimostrare: $T(n) = \Theta(1) + T(\frac{n}{2}) + \Theta(1) \leq c \cdot \log(n)$
  3. Considero vero per ipotesi di induzione $T(\frac{n}{2}) \leq c \cdot \log(\frac{n}{2})$ in quanto $\frac{n}{2} < n$ e sostituisco nella (2) ottenendo : $T(n) \leq c \cdot \log(\frac{n}{2}) + \Theta(k) = c \cdot \log(n) - c\log(2) + \Theta(k) \leq c\log(n)$

Metodo di sostituzione

Esempio 2

- Determiniamo un limite superiore per $T(n) = 2T(\frac{n}{2}) + n$
- Intuiamo $\mathcal{O}(n \log(n))$, dimostriamo $T(n) \leq c(n \log(n))$
- Supponiamo vero (hp. induzione) $T(\frac{n}{2}) \leq c(\frac{n}{2} \log(\frac{n}{2}))$
- Sostituiamo ottenendo che $T(n) \leq 2c(\frac{n}{2} \log(\frac{n}{2})) + n \leq cn \log(\frac{n}{2}) + n = cn \log(n) - cn \log(2) + n = cn \log(n) + (1 - c \log(2))n$
  - Il comportamento asintotico è quello che vorrei
- Riesco a trovare un n_0 opportuno dal quale in poi valga la diseguaglianza, assumendo che $T(1) = 1$ per definizione?
  - Provo $n_0 = 1$, ottengo $1 \leq 0 + 1 - c \log(2)$, no.
  - Con $n_0 = 3$ funziona, $T(3) = 2 \cdot 1 + 3 \leq 3c \log(3) + (1 - c \log(2))3$

Metodo di sostituzione

Esempio 2 - Un limite più stretto

- Determiniamo un limite superiore per $T(n) = 2T(\frac{n}{2}) + 1$
- Tentiamo di provare che è $\mathcal{O}(n)$, ovvero $T(n) \leq cn$
- Supponiamo vero (hp. induzione) $T(\frac{n}{2}) \leq c\frac{n}{2}$
- Sostituiamo ottenendo che $T(n) \leq 2c\frac{n}{2} + 1 = cn + 1$
  - Non possiamo trovare un valore di c che faccia rispettare l'ipotesi che vogliamo: $cn + 1$ è sempre maggiore di cn
- In questo caso, non siamo riusciti a dimostrare il limite tramite sostituzione
- N.B.: questo *non* implica che $T(n)$ non sia $\mathcal{O}(n)$
  - Prendere come ipotesi $T(n) \leq cn - b$, con b costante, consente di dimostrare che è $\mathcal{O}(n)$

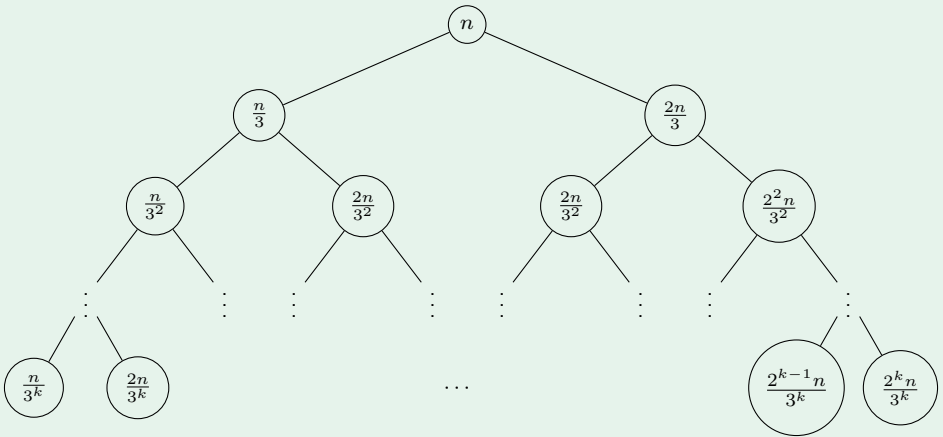
Metodo dell'albero di ricorsione

Espandere le chiamate ricorsive

- L'albero di ricorsione fornisce un aiuto per avere una congettura da verificare con il metodo di sostituzione, o un appiglio per calcolare la complessità esatta
- È una rappresentazione delle chiamate ricorsive, con la loro complessità
- Ogni chiamata costituisce un nodo in una sorta di albero genealogico: i chiamati appaiono come figli del chiamante
- Ogni nodo contiene il costo della chiamata, senza contare quello dei discendenti
- Rappresentiamo l'albero di $T(n) = T(\frac{n}{3}) + T(\frac{2n}{3}) + n$

Metodo dell'albero di ricorsione

Espandendo completamente



Metodo dell'albero di ricorsione

Espandendo completamente

- L'albero ha la ramificazione a profondità massima posta all'estrema destra del disegno precedente
- Sappiamo che essa ha profondità k che ricaviamo ponendo $\frac{2^k}{3^k}n = 1$ (il k-esimo pronipote a dx)
  $\rightarrow 2^k n = 3^k \rightarrow \log_3(2^k n) = k = \log_3(2^k) + \log_3(n) = \log_3(n) + \frac{\log_2(2^k)}{\log_2(3)}$ da cui abbiamo che $(\log_2(3) - 1)k = \log_3(n) \rightarrow k = c\log_3(n)$
- Il costo pessimo per il contributo di un dato livello è l'n del primo livello
- Congetturiamo che $T(n) = \Theta(n\log(n))$
  - Dimostriamolo mostrando che $T(n) = \mathcal{O}(n\log(n))$ e $T(n) = \Omega(n\log(n))$

Metodo dell'albero di ricorsione

$T(n) = \mathcal{O}(n \log(n))$

- Per hp. di induzione abbiamo sia che $T(\frac{n}{3}) \leq c_1(\frac{n}{3}\log(\frac{n}{3}))$ sia che $T(\frac{2n}{3}) \leq c_2(\frac{2n}{3}\log(\frac{2n}{3}))$ (dato che $\frac{2}{3}n < n$ e $\frac{1}{3}n < n$)
- Sostituendo abbiamo $T(n) \leq c_1(\frac{n}{3}\log(\frac{n}{3})) + c_2(\frac{2n}{3}\log(\frac{2n}{3})) + n = c_1(\frac{n}{3}(\log(n) - \log(3)) + c_2(\frac{2n}{3}(\log(n) - \log(3) + \log(2))) + c_3 n = c_4 n \log(n) - c_5 n + c_3 n \leq c_4 n \log(n)$ per una scelta opportuna delle costanti c_4, c_5, c_6

$T(n) = \Omega(n \log(n))$

- Hp ind. $T(\frac{n}{3}) \geq c_1(\frac{n}{3}\log(\frac{n}{3}))$, $T(\frac{2n}{3}) \geq c_2(\frac{2n}{3}\log(\frac{2n}{3}))$
- Sostituendo $T(n) \geq c_4 n \log(n) - c_5 n + c_6 n \geq c_4 n \log(n)$

Teorema dell'esperto (Master theorem)

Uno strumento efficace per le ricorsioni

- Il teorema dell'esperto è uno strumento per risolvere buona parte delle equazioni alle ricorrenze.
- Affinchè sia applicabile, la ricorrenza deve avere la seguente forma: $T(n) = aT(\frac{n}{b}) + f(n)$ con $a \geq 1, b > 1$
- L'idea di fondo è quella di confrontare $a^{\log_b(n)} = a^{\frac{\log_a(n)}{\log_a(b)}} = n^{log_b(a)}$ (costo totale delle foglie dell'AdR) con $f(n)$ (il costo della sola radice dell'AdR)
- Le ipotesi del teorema dell'esperto sono le seguenti:
  - a deve essere costante e $a \geq 1$ (almeno 1 sotto-problema per chiamata ricorsiva)
  - $f(n)$ deve essere sommata, non sottratta o altro a $aT(\frac{n}{b})$
  - Il legame tra $n^{log_b(a)}$ e $f(n)$ deve essere polinomiale
- Se queste ipotesi sono valide, è possibile ricavare informazione sulla complessità a seconda del caso in cui ci si trova

Master Theorem

Caso 1

- Nel primo caso $f(n) = \mathcal{O}(n^{log_b(a)-\epsilon})$ per un qualche $\epsilon > 0$
- La complessità risultante è $T(n) = \Theta(n^{log_b(a)})$
- Intuitivamente: il costo della ricorsione "domina" quello della singola chiamata
- Esempio: $T(n) = 9T(\frac{n}{3}) + n$
- Confrontiamo: $n^1 = n^{\log_3(9)-\epsilon} \Rightarrow \epsilon = 1$ ✓
- Otteniamo che la complessità è: $\Theta(n^{\log_3(9)}) = \Theta(n^2)$

Master Theorem

Caso 2

- Nel secondo caso abbiamo che $f(n) = \Theta(n^{log_b(a)})$
- La complessità risultante della ricorrenza è $T(n) = \Theta(n^{log_b(a)} \log(n))$
- Intuitivamente: il contributo della ricorsione e quello della singola chiamata differiscono per meno di un termine polinomiale
- Esempio: $T(n) = T(\frac{n}{3}) + \Theta(1)$
- Confrontiamo: $\Theta(1) = \Theta(n^{\lfloor log_3(1) \rfloor})$ è vero ?
  - Sì: $\Theta(1) = \Theta(n^0)$ ✓
- La complessità risultante è $\Theta(n^{log_3(1)} log(n)) = \Theta(log(n))$

Master Theorem

Caso 3

- In questo caso abbiamo che $f(n) = \Omega(n^{log_b(a)+\epsilon})$, $\epsilon > 0$
- Cond. Necessaria: vale che: $af(\frac{n}{b}) < cf(n)$ per un qualche valore di $c < 1$
- Se le ipotesi sono rispettate, abbiamo che $T(n) = \Theta(f(n))$
- Intuitivamente: il costo della singola chiamata è più rilevante della ricorsione
- Esempio: $T(n) = 8T(\frac{n}{3}) + n^3$
- Confrontiamo $n^3 = \Omega(n^{log_3(8)+\epsilon}) \Rightarrow \epsilon = 3 - log_3(8) > 0$ ✓
- Controlliamo se $8f(\frac{n}{3}) = \frac{8}{3^3}n^3 < cn^3$ per un qualche $c < 1$?
  - Sì, basta prendere c in $(1 - (\frac{8}{3^3}); 1)$ ✓
- La complessità dell'esempio è: $\Theta(n^3)$

Ordinare una collezione di oggetti

Un problema ricorrente

- Tra i problemi che capita più spesso di dover risolvere, l'ordinamento di una collezione di oggetti è un classico
- Un punto chiave dell'utilità dell'ordinamento è consentire di utilizzare una ricerca binaria sulla collezione ordinata
- Analizziamo soluzioni diverse considerando la loro complessità temporale, spaziale e relative peculiarità
- Proprietà di stabilità: in breve, un algoritmo di ordinamento è stabile se non cambia di ordine elementi duplicati

Insertion Sort

Ordinamento per inserimento di interi (ordine crescente)

INSERTIONSORT(A)

```
1  for i ← 2 to A.length
2      tmp ← A[i]
3      j ← i − 1 // ho salvato l' elemento in A[j + 1]
4      while j ≥ 1 and A[j] > tmp
5          A[j + 1] ← A[j] // sposto in avanti l' elemento se più grande di tmp
6          j ← j − 1
7      A[j + 1] ← tmp
```

- Raziocinio: seleziono un elemento e lo reinserisco nella porzione di vettore già ordinato, al suo posto
- $T(n)$: caso ottimo $\Theta(n)$, caso pessimo $\Theta(n^2)$, in gen. $\mathcal{O}(n^2)$. Complessità spaziale $\Theta(1)$. Stabile (usando $>$ non $\geq$).

Più veloce di $\mathcal{O}(n^2)$

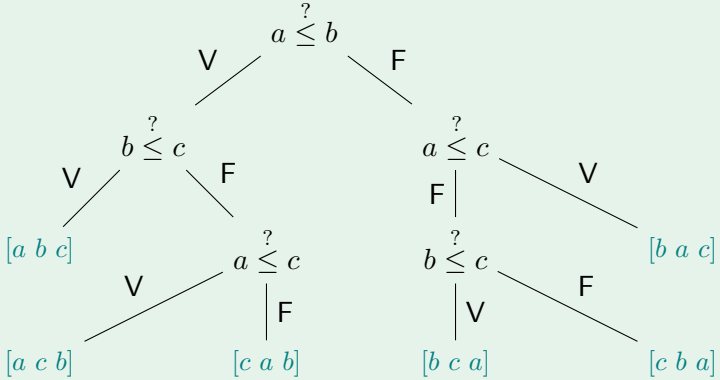
Limiti inferiori della complessità dell'ordinamento

- Abbiamo visto che nel caso pessimo l'Insertion sort è $\Theta(n^2)$
- E'possibile concepire un algoritmo più veloce? Sì
- Qual è il limite di complessità dell'ordinamento per confronto
  - È facile notare che qualunque procedura di ordinamento per n elementi è $\Omega(n)$
  - Sicuramente l'ordinamento è $\mathcal{O}(n^2)$: abbiamo l'insertion sort
- Astraiamo dalla specifica strategia di ordinamento: contiamo le azioni di confronto e scambio

Più veloce di $\mathcal{O}(n^2)$

Limiti inferiori della complessità dell'ordinamento

- Esaminiamo le decisioni per ordinare un vettore $[a\ b\ c]$



Limiti inferiori della complessità dell'ordinamento

Stima del numero di confronti

- L'albero costruito ha tante foglie quante permutazioni del vettore da ordinare
  - Per un vettore lungo n esso ha $n!$ foglie
- Assumiamo che la struttura sia la più compatta possibile
  - non ho confronti ridondanti tra elementi
- La lunghezza del più lungo dei percorsi radice-foglia è il numero max di confronti che devo fare per ordinare un vettore
- L'altezza dell'albero in questo caso è $\log_2$ del numero delle sue foglie $\rightarrow$ $\log(n!) \approx n\log(n) - \log(e)n + \mathcal{O}(\log_2(n))$
- La complessità migliore ottenibile è $\mathcal{O}(n\log(n))$

Merge Sort

Un algoritmo $\Theta(n \log(n))$

- Per avere un algoritmo di ordinamento con complessità di caso pessimo ottima, applichiamo una strategia *divide et impera*
- Suddividiamo il vettore di elementi da ordinare in porzioni più piccole, fin quando non sono ordinabili in $\Theta(1)$, dopodichè ri-assembliamo i risultati ottenuti
  - È importante che ri-assemblare i risultati ottenuti non abbia complessità eccessiva
- Analizziamo quindi la complessità di fondere due array ordinati in un unico array, anch'esso ordinato
  - Consideriamo i due array come slices di un unico array `A`: `A[p..q]`, `A[q+1..r]`

Fusione di `A[p..q]`, `A[q+1..r]` in `A[p..r]`

```
MERGE(A, p, q, r)
 1  len_1 ← q − p + 1
 2  len_2 ← r − q
 3  ALLOCA(L[1..len_1 + 1])
 4  ALLOCA(R[1..len_2 + 1])
 5  for i ← 1 to len_1 // Copia della prima metà
 6        L[i] ← A[p + i − 1]
 7  for i ← 1 to len_2 // Copia della seconda metà
 8        R[i] ← A[q + i]
 9  L[len_1 + 1] ← ∞;   R[len_2 + 1] ← ∞ // sentinelle
10  i ← 1;   j ← 1;
11  for k ← p to r
12        if L[i] ≤ R[j]
13              A[k] ← L[i]; i ← i + 1
14        else
15              A[k] ← R[j]; j ← j + 1
```

Merge

Analisi di complessità

- L'algoritmo alloca due array ausiliari, grossi quanto le parti da fondere, più alcune variabili ausiliarie in numero fissato
  - Complessità spaziale $\Theta(n)$
- Tralasciando le porzioni sequenziali, l'algoritmo è composto da 3 cicli:
  - Due per copiare le parti da fondere: complessità $\Theta(n)$
  - Uno che copia in A gli elementi in ordine: complessità $\Theta(n)$
- In totale abbiamo che MERGE è $\Theta(n)$

MergeSort

Algoritmo

$\textsc{MergeSort}(A, p, r)$

```
1  if p < r − 1
2        q ← ⌊(p+r)/2⌋
3        MergeSort(A, p, q)
4        MergeSort(A, q + 1, r)
5        Merge(A, p, q, r)
6  else // Caso base della ricorsione: ho solo ≤2 elementi
7        // N.B. se ho 1 elemento non devo fare nulla
8        if A[p] > A[r]
9              tmp ← A[r]
10             A[r] ← A[p]
11             A[p] ← tmp
```

- Costo: $T(n) = 2T(\frac{n}{2}) + \Theta(n)$: Caso 2 MT $\rightarrow \Theta(n\ \log(n))$

Quicksort

Un'alternativa divide-et-impera

- Quicksort ordina senza spazio ausiliario (sul posto, o *in place*)
- Quicksort applica il divide-et impera ad una slice `A[lo..hi]`:

Dividi Scegli un elemento `A[p]` (detto pivot) come punto di suddivisione di `A[lo..hi]` e sposta gli elementi di `A[lo..hi]` in modo che tutti quelli di `A[lo..p-1]` siano minori o uguali al pivot

Impera Ordina `A[lo..p-1]`, `A[p+1..hi]` con Quicksort

Combina Nulla! L'ordinamento è eseguito in place

$\textsc{Quicksort}(A, lo, hi)$

1 **if** $lo < hi$
2 $\quad p \leftarrow \textsc{Partition}(A, lo, hi)$
3 $\quad \textsc{Quicksort}(A, lo, p-1)$
4 $\quad \textsc{Quicksort}(A, p+1, hi)$

Quicksort

Schema di partizione di Lomuto

PARTITIONLOMUTO(A, lo, hi)

```
1  pivot ← A[hi]
2  i ← lo − 1
3  for j ← lo to hi − 1
4        if A[j] ≤ pivot
5              i ← i + 1
6              SCAMBIA(A[i], A[j])
7  SCAMBIA(A[i + 1], A[hi])
8  return i + 1
```

- i indica la posizione dell' ultimo elemento $\leq$ pivot, escluso il pivot stesso
- L' $(i+1)$-esimo elemento è nella sua posizione definitiva dopo PARTITIONLOMUTO, posso escluderlo nelle chiamate ricorsive
- Complessità di PARTITIONLOMUTO: $\Theta(n)$

Quicksort

Schema di partizione di Hoare

PARTITIONHOARE(A, lo, hi)

```
1   pivot ← A[lo]
2   i ← lo − 1; j ← hi + 1
3   while true
4       repeat
5           j ← j − 1
6       until A[j] ≤ pivot
7       repeat
8           i ← i + 1
9       until A[i] ≥ pivot
10      if i < j
11          SCAMBIA(A[i], A[j])
12      else return j
```

- Effettua $\frac{1}{3}$ degli scambi di Lomuto, in media (asint. $\Theta(n)$)
- N.B. la partizione di Hoare restituisce l'indice dell' ultimo elemento $\leq pivot$ (non necessariamente $= pivot$)
- serve una modifica a QUICKSORT

QUICKSORT(A, lo, hi)

```
1  if lo < hi
2      p ←PARTITIONHOARE(A, lo, hi)
3      QUICKSORT(A, lo, p)
4      QUICKSORT(A, p + 1, hi)
```

Quicksort

Complessità

- Il calcolo di PARTITION ha complessità temporale $\Theta(n)$, con n la lunghezza del vettore di cui deve operare la partizione
- La complessità dell'intero Quicksort risulta quindi $T(n) = T(\frac{n}{a}) + T(n - \frac{n}{a}) + \Theta(n)$, dove il valore a dipende da quanto "bene" PARTITION ha suddiviso il vettore
- Caso pessimo: il vettore è diviso in porzioni lunghe $n - 1$ e 1
  - La ricorrenza diventa $T(n) = T(n-1) + T(1) + \Theta(n)$
  - Si dimostra facilmente che è $\Theta(n^2)$
- Caso ottimo: il vettore è diviso in due porzioni lunghe $\frac{n}{2}$
  - La ricorrenza diventa $T(n) = 2T(\frac{n}{2}) + \Theta(n)$
  - È la stessa del MergeSort, $\Theta(n \log(n))$
- Caso medio: $\Theta(n \log(n))$ e la costante nascosta da Θ è $1,39$

Riassumendo

Un confronto tra ordinamenti per confronto

Algoritmo	Stabile?	$T(n)$ (caso pessimo)	$T(n)$ (caso ottimo)	$S(n)$
Insertion	✓	$\Theta(n^2)$	$\Theta(n)$	$O(1)$
Merge	✓	$\Theta(n \log(n))$	$\Theta(n \log(n))$	$\Theta(n)$
Quick	×	$\mathcal{O}(n^2)$	$\Omega(n \log(n))$	$O(1)$

- Non è possibile essere più veloci usando algoritmi di ordinamento *per confronto*
- C'è modo di fare meglio ordinando senza confrontare tra elementi?

Algoritmi non comparativi

Ordinare senza confrontare

- Il vincolo che abbiamo sulla complessità minima è legato al fatto che confrontiamo gli elementi da ordinare *tra loro*
- Nel caso in cui possiamo fare assunzioni sulla distribuzione o sul dominio degli elementi da ordinare, possiamo fare a meno dei confronti!
- Vediamo un esempio di algoritmo di ordinamento senza confronti il *counting sort*
  - Assunzione: il dominio degli elementi è *finito* e di dimensioni "ragionevoli" (dovremo rappresentarlo per esteso)
  - Intuizione: ordino calcolando l'istogramma delle frequenze e stampandone gli elementi in ordine

Counting Sort

Versione non stabile, k valore massimo degli el. di A

CountingSort(A)

```
1  Ist[0..k] ← 0 // Nota: costo Θ(k)
2  for i ← 0 to A.length − 1
3        Ist[A[i]] ← Ist[A[i]] + 1
4  idxA ← 0
5  for i ← 0 to k
6        while Ist[i] > 0
7              A[idxA] ← i
8              idxA ← idxA + 1
9              Ist[i] ← Ist[i] − 1
```

- La complessità temporale è dominata dal ciclo alle righe 5–8: $\mathcal{O}(n + k)$
- Se $k \gg n$ la complessità in pratica può essere molto alta

Counting Sort, versione stabile

Versione *stabile*: strategia

- Il counting sort stabile parte con il calcolare il numero delle occorrenze di ogni elemento come quello classico
- A partire dall'istogramma delle frequenze Ist, lo trasforma nel vettore contenente il conteggio degli elementi con valori $\leq$ di quello dell'indice del vettore
- Calcolato ciò, piazza un elemento calcolando la sua posizione come il valore corrente dell'informazione cumulativa contenuta in Ist
- L'informazione cumulativa è decrementata: effettivamente esiste un elemento in meno $\leq$ all'indice del vettore

Counting Sort

Versione *stabile*, out-of-place, k valore massimo degli el. di A

COUNTINGSORT(A)

```
1  B[0..A.length − 1] ← 0
2  Ist[0..k] ← 0 // Nota: costo Θ(k)
3  for i ← 0 to A.length − 1 // Calcola istogramma
4        Ist[A[i]] ← Ist[A[i]] + 1
5  sum ← 0
6  for i ← 0 to k // calcola num. elem. ≤ i
7        sum ← sum + Ist[i]
8        Ist[i] ← sum
9  for i ← A.length − 1 to 0
10       idx ← Ist[A[i]]
11       B[idx − 1] ← A[i]
12       Ist[A[i]] ← Ist[A[i]] − 1
13 return B
```

Strutture dati - Parte 1

Dipartimento di Elettronica, Informazione e Bioingegneria
Politecnico di Milano

17 maggio 2023

Strutture dati

Organizzare i dati processati

- Algoritmi efficienti necessitano di poter accedere, modificare, cancellare i dati da una collezione in modo efficiente
- L'unico modo che abbiamo visto finora di organizzare elementi su cui effettuiamo un calcolo (nello pseudocodice) sono i *vettori*
- Analizziamo come è possibile rappresentare collezioni di elementi in modo più organizzato $\rightarrow$ strutture dati più evolute
- Queste strutture possono usare etichette per identificare un oggetto: le etichette sono dette *chiavi*
- Analizziamo l' efficienza delle strutture dati quantificando quella delle operazioni che effettuiamo su di esse

Operazioni tipiche su strutture dati

Interrogare la struttura

- `Search(S,k)`: restituisce il riferimento a e con `e.key`=k in S, NIL se k non è contenuto in S
- `Minimum(S)`: se le chiavi sono ordinate, restituisce e con la chiave minima
- `Maximum(S)`: come sopra, ma la chiave ha valore massimo
- `Successor(S,k)`: restituisce e t.c. `e.key` è la più piccola tra le chiavi $>$k
- `Predecessor(S,k)`: come sopra, ma considerando il predecessore

Modificare la struttura

- `Insert(S,e)`: Inserisce un oggetto e nella struttura
- `Delete(S,e)`: Cancella un oggetto e dalla struttura

Analisi critica di strutture note

Vettori

- Un vettore è una struttura dati compatta in memoria in cui si accede direttamente ad ogni elemento, data la sua posizione
- Se il vettore di lunghezza n non è ordinato:
  - ricerca, minimo, massimo, successore sono $\mathcal{O}(n)$
  - inserimento e cancellazione costano $\mathcal{O}(n)$ (la cancellazione può essere ridotta a $\mathcal{O}(1)$ usando dei simboli di "cella vuota")
- Se il vettore di lunghezza n è ordinato:
  - minimo e massimo:$\Theta(1)$, ricerca e successore $\Theta(\log(n))$
  - inserimento e cancellazione costano $\mathcal{O}(n)$
- Inserimenti in vettore pieno:
  - sono rifiutati (tenendo un conteggio degli elementi):$\mathcal{O}(1)$
  - causano una riallocazione $\mathcal{O}(n)$: (causa copia degli elementi esistenti)

Analisi critica di strutture note

Liste semplicemente connesse

- Una lista semplice stocca gli elementi sparsi in memoria: ogni elemento ha un riferimento al successivo (i.e., puntatore)
- Se la lista di lunghezza n non è ordinata:
  - ricerca, minimo, massimo, successore sono $\mathcal{O}(n)$
  - inserimento: $\mathcal{O}(1)$, cancellazione: $\mathcal{O}(n)$ se l'elemento va trovato, $\mathcal{O}(1)$ se si ha un riferimento alla posizione di inserimento
- Se la lista di lunghezza n *è* ordinata:
  - uno dei due tra minimo e massimo è $\Theta(1)$ l'altro $\Theta(n)$
    - Con puntatore accessorio all'ultimo elemento: entrambi $\Theta(1)$
  - ricerca e successore sono $\mathcal{O}(n)$
  - inserimento: $\mathcal{O}(n)$, cancellazione: $\mathcal{O}(n)$

Pile (Stack)

Una struttura dati familiare

- Una pila è una struttura dati con le seguenti operazioni:
  - `Push(S,e)`: aggiunge l'elemento in cima alla pila
  - `Pop(S)`: restituisce l'elemento in cima alla pila cancellandolo
  - `Empty(S)`: restituisce `true` se la pila è vuota
- Questa struttura dati astratta può essere realizzata usando una lista semplicemente connessa o un vettore

Realizzazione con lista

- Lo stoccaggio dati è nella lista, le operazioni diventano:
  - `Push(S,e)`: inserisci in testa alla lista $\mathcal{O}(1)$
  - `Pop(S)`: restituisci il primo elemento della lista, cancellandolo dalla stessa $\mathcal{O}(1)$
  - `Empty(S)`: controlla se il successore della testa è NIL: $\mathcal{O}(1)$

Pile (Stack)

Realizzazione con vettore

- Lo stoccaggio dati è nella celle del vettore, viene mantenuto l'indice della cima della pila (Top of Stack, ToS)
  - `Push(S,e)`: se c'è spazio, Incrementa ToS, salva e in `A[ToS]`: $\mathcal{O}(1)$; se manca spazio rifiuta $\mathcal{O}(1)$ o rialloca $\mathcal{O}(n)$
  - `Pop(S)`: restituisci `A[ToS]` corrente, decrementa `ToS`: $\mathcal{O}(1)$
  - `Empty(S)`: Restituisci `ToS` $\stackrel{?}{=} 0$: $\mathcal{O}(1)$
- Nessun vantaggio (dal punto di vista astratto) rispetto all'implementazione a pila, (uno svantaggio se si rialloca)
  - In pratica, avere dati non coesi in memoria penalizza le caches
  - $\rightarrow$ può valer la pena di usare un vettore se non ci sono troppe riallocazioni (e.g., con una preallocazione ragionata)

Code (Queues)

Struttura ed operazioni

- Una coda è una struttura dati con le seguenti operazioni:
  - `Enqueue(Q,e)`: aggiunge e alla fine della coda
  - `Dequeue(Q)`: restituisce l'elemento all'inizio della coda, cancellandolo dalla stessa
  - `Empty(Q)`: restituisce `true` se la coda è vuota
- Come nel caso della pila, è possibile realizzare una coda sia con una lista che con un vettore

Code (Queues)

Realizzazione con vettore

- Salviamo i dati in un vettore A, lungo l, indice del primo elemento 0
- Teniamo traccia della posizione dove va inserito un nuovo elemento e di quella dell'elemento più vecchio con due indici `tail` e `head` e del numero di elementi contenuti n
- Gli indici vengono incrementati $\bmod l$
  - `Enqueue(Q,e)`: se $n < l$, inserisci l'elemento in A[`tail`], incrementa n e `tail`: $\mathcal{O}(1)$, altrimenti segnala l'errore, $\mathcal{O}(1)$
  - `Dequeue(Q)`: se $n > 0$, restituisci `A[head]` corrente, decrementa n, incrementa `head`: $\mathcal{O}(1)$
  - `Empty(S)`: Restituisci $n \stackrel{?}{=} 0$: $\mathcal{O}(1)$
- Per ampliare lo stoccaggio: allocazione fresca e copia degli elementi estraendoli con `Dequeue(Q)`: $\Theta(n)$

Code (Queues)

Realizzazione con lista

- Lo stoccaggio dei dati è effettuato negli elementi della lista
- Teniamo traccia dell'ultimo elemento della lista (oltre al primo) con un puntatore `tail`
  - `Enqueue(Q,e)`: inserisci l'elemento e in coda alla lista, aggiornando `tail`: $\mathcal{O}(1)$
  - `Dequeue(Q)`: restituisci l'elemento in testa se diverso da NIL, cancellandolo e aggiornando `head`: $\mathcal{O}(1)$
  - `Empty(S)`: Restituisci `head`$\stackrel{?}{=}$`tail`: $\mathcal{O}(1)$

Mazzo o coda a due fini (Deque)

Struttura dati

- La struttura dati si comporta come un mazzo di carte, di cui ognuna contiene un elemento
- E'possibile aggiungere sia in testa che in coda alla struttura:
  - `PushFront(Q,e)`: inserisci l'elemento e in testa al mazzo
  - `PushBack(Q,e)`: inserisci l'elemento e in coda al mazzo
  - `PopFront(Q)`: restituisci l'elemento in testa, cancellandolo
  - `PopBack(Q)`: restituisci l'elemento in testa, cancellandolo
  - `Empty(S)`: Restituisci `true` se il mazzo è vuoto

Mazzo o coda a due fini (Deque)

Realizzazione con vettore

- Lo stoccaggio dei dati è effettuato in modo analogo alla coda semplice
- `PushBack` e `PopFront` si comportano come `Enqueue` e `Dequeue` della coda realizzata con un vettore
  - `PopBack(Q)`: se $n > 0$, restituisci `A[tail]` corrente, decrementa n, decrementa `tail`: $\mathcal{O}(1)$
  - `PushFront(Q,e)`: se $n < l$, decrementa e `head`, inserisci l'elemento in $A[\texttt{head}]$, incrementa n : $\mathcal{O}(1)$, altrimenti segnala l'errore, $\mathcal{O}(1)$
  - `Empty(S)`: Restituisci $n \stackrel{?}{=} 0$: $\mathcal{O}(1)$
- Ampliamento dello stoccaggio: come per la coda

Mazzo o coda a due fini (Deque)

Realizzazione con lista *doppiamente* concatenata

- Un elemento di una lista doppiamente concatenata ha puntatori sia al precedente che al seguente.
- Un modo comune di rappresentarne una vuota è una coppia di elementi, `head` e `tail` che puntano l'uno all'altro
- `PushBack` e `PopFront` si comportano come `Enqueue` e `Dequeue` di una coda realizzata con una lista
  - `PopBack(Q)`: restituisci `tail.prev` corrente se diverso da `head`, rimuovendolo dalla lista: $\mathcal{O}(1)$
  - `PushFront(Q,e)`: aggiungi l'elemento in testa, aggiornando head e il suo successore $\mathcal{O}(1)$
  - `Empty(S)`: Restituisci `head.next`$\stackrel{?}{=}$`tail` in $\mathcal{O}(1)$
- Ampliamento dello stoccaggio: come per la coda

Riassumendo

Strutture dati lineari

- Tutte le operazioni viste su pile, code e mazzi sono $\mathcal{O}(1)$ nel caso delle implementazioni basate su lista, o su vettore con stoccaggio finito
- Le implementazioni che utilizzano vettori come stoccaggio e consentono di ampliarli nel caso vi sia necessità pagano un costo lineare per l'ampliamento

Liste doppiamente concatenate

- Si comportano come le liste semplici, tranne la cancellazione
- Cancellare un elemento fornito alla `Delete(L,e)` per riferimento è $O(1)$: $e.prev.next \leftarrow e.next$; $e.next.prev \leftarrow e.prev$

Dizionari

Rappresentare collezioni di oggetti

- Un dizionario è una struttura dati astratta che contiene elementi accessibili direttamente, data la loro chiave
- Offerto da alcuni linguaggi di programmazione come tipo base (Python)
- Nei nostri esempi, assumiamo che le chiavi siano numeri naturali
  - Se non lo sono, è sufficiente considerare la loro rappresentazione binaria come il corrispettivo numero
- Le operazioni supportate da un dizionario sono `Insert`, `Delete` e `Search`
- E'possibile implementare un dizionario con diverse strutture dati concrete

Dizionari

Un primo approccio

- Nel caso in cui le *possibili* chiavi siano un numero limitato un'implementazione di un dizionario è un vettore di puntatori
- Le chiavi vengono usate come indice del vettore
- Le operazioni sul dizionario sono implementate come:
  - `Insert(D,e)`: `D[e.key]` $\leftarrow$ `e`
  - `Delete(D,e)`: `D[e.key]` $\leftarrow$ `NIL`
  - `Search(D,e.key)`: **return** `D[e.key]`
- Complessità computazionale: $\Theta(1)$ per tutte le azioni
- Complessità spaziale: $\mathcal{O}(|\mathbf{D}|)$, con $\mathbf{D}$ il dominio delle chiavi
  - Estremamente oneroso se il dominio è molto ampio

Tabelle Hash

Maggior efficienza in spazio

- Una tabella hash implementa un dizionario con una complessità in memoria pari al numero di chiavi m per cui è *effettivamente presente* un valore
  - Il dominio delle chiavi $\mathbf{D}$ può essere arbitrariamente grande/infinito
- Approccio tipico: prealloco spazio per m chiavi
  - Rialloco solo quando devo inserire più di m chiavi
- Idea principale: uso come indice (indirizzo) della tabella il risultato del calcolo di una funzione della chiave detta funzione di hash $h(k)$
  - $h(\cdot) : \mathbf{D} \to \{0, \ldots, m-1\}$ serve a trasformare una chiave arbitraria in un indirizzo ammissibile (tra 0 e $m-1$)

Tabelle Hash

Efficienza

- Se il calcolo di $h(\cdot)$ è $\mathcal{O}(1)$ la tabella di hash ideale ha la stessa efficienza temporale del dizionario realizzato con un vettore di $|\mathbf{D}|$ puntatori

Il problema delle collisioni

- Idealmente, h mappa ogni chiave su di un distinto elemento del suo codominio
  - Impossibile! Per costruzione $|\mathbf{D}| \gg m$ (specie se $|\mathbf{D}| = \infty$)
- Chiamiamo *collisione* i casi in cui, date due chiavi k_1, k_2; con $k_1 \neq k_2$ abbiamo che $h(k_1) = h(k_2)$
  - Dobbiamo gestire le collisioni, oppure due elementi con chiavi distinte finiranno allo stesso indirizzo

Gestione delle collisioni

Indirizzamento chiuso (a.k.a. open hashing, chaining)

- Ogni riga della tabella (bucket) contiene la testa di una lista al posto del puntatore ad un singolo elemento
- Nel caso di collisione, l'elemento nuovo viene aggiunto in testa alla lista ($\Theta(1)$)
- Per cercare/cancellare un elemento di chiave k, è necessario cercare nell'intera lista di quelli del bucket $h(k)$

Efficienza computazionale

Stime di costo

- Caso pessimo: tutti gli elementi collidono dando origine ad una lista (open hashing) lunga m elementi: `Insert`/`Delete`/`Search` in $\mathcal{O}(m)$
- Chiamiamo fattore di carico $\alpha = \frac{n}{m}$, $0 \leq \alpha \leq \frac{|\mathbf{D}|}{m}$
  - α può essere più grande di $1 \rightarrow$ tutti i bucket sono pieni

Ipotesi di Hashing Uniforme Semplice (IHUS)

- Una opportuna scelta di h fa sì che ogni chiave abbia la stessa probabilità $\frac{1}{m}$ di finire in una qualsiasi delle m celle
- Come fare "la scelta opportuna"? Dipende dalla distribuzione delle chiavi

Efficienza computazionale

Stime di costo

- Assumiamo l'IHUS, per l'open hashing abbiamo che:
  - La lunghezza media di una lista è il fattore di carico
  - Il tempo medio per cercare una chiave non presente è $\Theta(1+\alpha)$
  - Il tempo medio per cercare una chiave presente è sempre $\Theta(1+\alpha)$
    - risultato del calcolo del valor medio del numero di oggetti aggiunti al bucket di x dopo che x è stato inserito $1+\frac{\alpha}{2}-\frac{\alpha}{2n}$
- Se il fattore di carico non è eccessivo tutte le operazioni sono $\mathcal{O}(1)$ in media
  - In pratica: si decide un fattore di carico e si rialloca quando la tabella eccede

Gestione delle collisioni - 2

Indirizzamento aperto (a.k.a., open addressing, closed hashing)

- In caso di collisione si seleziona secondo una regola deterministica l'indirizzo di un altro bucket di destinazione (procedimento di ispezione)
- Nel caso non si trovino bucket vuoti:
  - L'inserimento semplicemente fallisce $\Theta(m)$
  - Si rialloca una tabella più grande, vuota, e si ri-inseriscono tutti gli elementi della vecchia nella nuova (ricalcolando la loro hash, re-hashing), incluso il nuovo $\Theta(n)$
- Si modifica la procedura di ricerca, affinchè, se l'elemento non viene trovato nel suo bucket, essa effettui la stessa ispezione
- La cancellazione è effettuata inserendo un opportuno valore (tombstone) che non corrisponde ad alcuna chiave

Procedure di ispezione

Ispezione lineare (Linear probing) e clustering

- Il metodo di ispezione più semplice è l'*ispezione lineare*
  - Dato $h(k, 0) = a$ il bucket dove avviene la collisione al primo ($i = 0$) tentativo di inserimento, si sceglie $h(k, i) = a + c \cdot i \bmod m$ come bucket candidato per l'i-esimo inserimento
- Problema: se ci sono molte collisioni su un dato bucket, peggiorerà la probabilità di collisione in tutte le vicinanze
  - Il fenomeno è detto di *clustering primario* delle collisioni
  - Per alcune scelte di h, il peggiorare delle prestazioni dovuto al clustering dell'ispezione lineare è molto forte
  - É possibile avere clustering di dimensione logaritmica nella dimensione della tabella, effettuando rehashing molto prima che sia piena

Procedure di ispezione

Ispezione quadratica (Quadratic probing)

- Per mitigare il fenomeno del clustering è possibile utilizzare il criterio di ispezione quadratica: $h(k, i) = a + c_1 i + c_2 i^2 \bmod m$
  - Viene evitato il clustering banale nell'intorno di alcuni elementi
  - Non è più garantito a priori che la sequenza di ispezione tocchi tutte le celle: potrei dover fare rehashing a tabella non piena
- Rimuove il clustering primario
- Chiavi con la stessa posizione iniziale generano ancora clustering: hanno la stessa sequenza di ispezione (clustering secondario)

Una sequenza ideale

Per tabelle di dimensione $m = 2^x$

- Lemma: $h(k,i) = a + \frac{1}{2}i + \frac{1}{2}i^2$ genera tutti i valori in $[0, m-1]$
- Dimostrazione: (Per assurdo) Esistono $0 < p < q < m-1$ tali che $\frac{1}{2}p + \frac{1}{2}p^2 = \frac{1}{2}q + \frac{1}{2}q^2 \bmod m \Rightarrow p + p^2 = q + q^2 \bmod 2m$
- Fattorizzando abbiamo $(q-p)(p+q+1) = 0 \bmod 2m$
  - Se $(q-p) = 0 \bmod 2m \Rightarrow q = p$ ↯
  - $(p+q+1) = 0 \bmod 2m$: dati i range possibili $0<p<q<m-1$ la somma è $\in [1, 2m-1]$ ↯
  - $(q-p)(p+q+1) = 0 \bmod 2m$, ma $(q-p) \neq 0 \bmod 2m$ e $(p+q+1) \neq 0 \bmod 2m$. Osserviamo che almeno uno tra $(q-p)$ e $(p+q+1)$ è dispari
  - Essendo $m = 2^x$ solo il fattore pari può essere multiplo di $2m$, ma $(q-p) \leq m-1$ e $(p+q+1) \leq 2m-2$ ↯ ■

Doppio Hashing

Un'ispezione dipendente dalla chiave

- Definiamo $h(k, i) = h_1(k) + h_2(k)i \bmod m$: il passo di ispezione dipende dalla chiave
- Per essere sicuro di ispezionare tutti i bucket, $h_2(k)$ deve essere coprimo con m :
  - Per $m = 2^x$ basta fare sì che h_2 generi solo numeri dispari
  - Se m è primo, basta fare sì che h_2 generi un numero $< m$
  - N.B.: h_2 non deve mai dare zero, altrimenti la sequenza di ispezione degenera

Stime di costo

Closed hashing

- Il tempo per trovare un elemento dipende anche dalla sequenza di ispezione
- Ipotesi di hashing uniforme: generalizziamo la IHUS dicendo che tutte le sequenze di ispezione sono equiprobabili
- Il fattore di carico è sempre: $0 \leq \alpha \leq 1$ (al massimo un oggetto per bucket)
- Se consideriamo la v.a. $\mathcal{X}$ che modella il numero di passi di ispezione fatti senza trovare il valore desiderato, abbiamo $\Pr(\mathcal{X} \geq i) \leq \alpha^{i-1}$ il cui valor medio su i è $\frac{1}{1-\alpha}$: cercare un valore *non presente* costa $\frac{1}{1-\alpha}$ (in media)
- Il numero medio di tentativi prima di *trovare* un elemento desiderato è ricavato assumendo di trovarlo al j-esimo tentativo e mediando il numero di insuccessi su tutte le n chiavi presenti in tabella: si ottiene $\frac{1}{\alpha}\log(\frac{1}{1-\alpha})$

Possibili funzioni di hash

Metodo della divisione

- Un metodo semplice è $h(k) = k \bmod m$
  - Distribuzione di $h(k)$ non uniforme in $\{0, \ldots, m-1\}$
  - Va evitato se $m = 2^i$: $h(k)$ dipende solo dai bit meno significativi
- Distrib. (quasi) uniforme: m primo, vicino ad una potenza di 2
  - Primi di Fermat: hanno forma $2^i + 1$, e.g., $17, 257, 65537$
  - Primi di Mersenne: hanno forma $2^i - 1$, e.g.,$127, 8191, 131071$

Possibili funzioni di hash

Metodo della moltiplicazione

- Un metodo semplice è $h(k) = \lfloor m(\alpha k - \lfloor \alpha k \rfloor) \rfloor$ con α scelto costante $\in \mathbb{R}$
  - n.b.: $\alpha k - \lfloor \alpha k \rfloor$ è la parte frazionaria di αk
- In questo caso, la dimensione della tabella m non è critica
  - Spesso $m = 2^x$ in modo da effettuare le moltiplicazioni con un semplice shift
- Una scelta possibile per α è $\frac{\sqrt{5}-1}{2}$, rappresentato a virgola fissa (proposto da Knuth): dà buoni risultati in pratica
- Un modo pratico di calcolare $h(k)$ in `C`, nota la larghezza di parola del calcolatore (e.g. 32b) è calcolare $2^{32}\alpha k$ su 64 bit, moltiplicare per k e troncare a 32 bit
  `(uint32_t)( k * (double)ALPHA * ((uint64_t)1 << 32))`
- La porzione in rosso è costante e può essere precalcolata

Hashing universale

Prevenire casi pessimi indotti

- Abbiamo finora presunto che le collisioni fossero accidentali
- Se vengono indotte da un utente che conosce la funzione di hash ed inserisce una sequenza di elementi ad-hoc? Accessi certi in $\mathcal{O}(n)$. Casi pratici: `PHP`, filesystem `ext3`/`ext4`
- Come evitarlo? Scegliendo una funzione di hash casualmente all'interno di una **famiglia** di buone funzioni
- Si può dimostrare che $h_{a,b}(k) = ((ak + b) \bmod p) \bmod m$ con p primo, $p > m$, per qualunque $a, b \in \mathbb{Z} \setminus 0$ distribuisce uniformemente le chiavi nella tabella
- É sufficiente scegliere casualmente a e b all'interno del programma, per ogni istanza della tabella

Hashing crittografico

Una tabella di hash, senza tabella

- Consideriamo il caso in cui il codominio di $h(\cdot)$ sia enorme (e.g. stringhe da 256 bit $\rightarrow$ codominio da 2^{256} elem.)
- Con una buona h (che rispetti l'IHUS), servono moltissime chiavi prima di vedere una collisione
- Non potrò mai materializzare la tabella di hash, ma l'hash di un valore funge da "etichetta unica" del valore stesso
- È importante che non si possano trovare collisioni o preimmagini (dato $h(k)$, trovare k) in tempo utile
- Caso pratico: `SHA-2-256`: $\mathbf{D} = \{0,1\}^{2^{64}}$, codominio $\{0,1\}^{256}$

Strutture dati - Parte 2

Dipartimento di Elettronica, Informazione e Bioingegneria
Politecnico di Milano

22 maggio 2024

Alberi

Una struttura dati flessibile

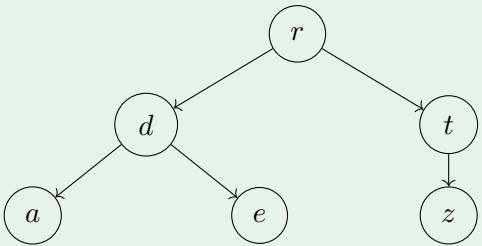
- Una struttura dati versatile è il cosiddetto *albero*
- Ne abbiamo già fatto uso informalmente (limite inferiore complessità dell'ordinamento per confronto)
- In estrema sintesi:
  - Un albero è costituito da un insieme di nodi e uno di archi che li collegano
  - Ogni nodo ha al più un arco entrante, ma un numero arbitrario di archi uscenti
- Gli alberi sono una rappresentazione efficiente per insiemi di dati ordinati

Alberi

Definizione

Un albero A è una coppia $(\mathbf{V}, \mathbf{E})$ dove $\mathbf{V}$ è un insieme di nodi e $\mathbf{E}$ un insieme di archi (coppie di nodi ordinate). Ogni nodo può apparire un'unica volta come destinazione di un arco (= secondo elemento della coppia). Non sono possibili cicli.

Graficamente



- $\mathbf{V} = \{r, a, d, e, t, z\}$
- $\mathbf{E} = \{(r, d), (r, t), (d, a), (d, e), (t, z)\}$

Alberi

Nomenclatura

- **Radice**: É l'unico nodo dell'albero privo di un arco entrante
- **Foglia**: Un nodo senza archi uscenti
- **Padre** (o genitore): di un nodo n: il nodo da cui l'arco entrante in n ha origine
- **Figlio** (o discendente): di un nodo n: il nodo in cui uno degli archi uscenti da n termina
- Un albero in cui ogni nodo ha al più due figli è detto *albero binario*
- Livello: distanza, in numero di archi, di un nodo dalla radice
- Albero completo: un albero in cui tutti i livelli hanno tutti i nodi tranne l'ultimo
  - i nodi dell'ultimo livello sono impacchettati da sinistra

Alberi binari

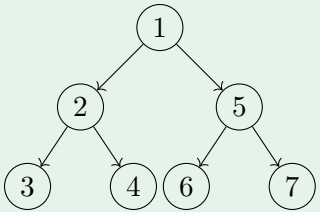
In pseudocodice

- Ci occuperemo di alberi binari (= ogni padre ha 2 figli)
- É utile dare una definizione ricorsiva di albero:
  - Un albero è formato da un nodo radice a cui sono collegati due alberi, il sottoalbero destro e quello sinistro
  - Un albero può essere vuoto (NIL)
- Le azioni sull'albero indicizzeranno i nodi con una chiave
  - Simile a quanto fatto per le tabelle hash, la chiave è un intero
- Dato un nodo `A`:
  - `A.left` è il riferimento al figlio sinistro, `A.right` al destro
  - `A.p` è il riferimento al padre, `A.key` è la chiave
- Ogni albero A ha un riferimento `A.root` alla radice
  - `A.p` è `NIL` solo per la radice

Stoccaggio con struttura dati implicita

Alberi binari stoccati con un vettore

- Un albero può essere materializzato in memoria naturalmente con una struttura basata su puntatori
- Alternativamente è possibile utilizzare un vettore per contenere le chiavi (efficiente se l'albero è completo)



1	2	5	3	4	6	7

- La radice dell'albero è stoccata nella prima posizione del vettore
- Dato un nodo contenuto in posizione i il suo figlio sx è in posizione $2i + 1$, il dx in $2i + 2$ (contando da 0 le pos.
- Il padre del nodo stoccato in posizione i (se esiste) si trova in posizione $\lfloor \frac{i-1}{2} \rfloor$

Alberi binari

Visita di un albero

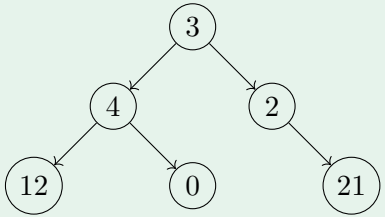
- Su di un albero è possibile effettuare operazioni di inserimento, ricerca e cancellazione di nodi come sulle altre strutture dati
- L'operazione caratteristica degli alberi è il cosiddetto *attraversamento* o *visita* per enumerare le chiavi contenute
- La definizione naturale degli algoritmi di visita è ricorsiva
- Il fattore discriminante tra le diverse strategie è l'*ordine* in cui i nodi vengono visitati

Visita di un albero

Visita *in-ordine* (*in-order*)

- Nella visita in ordine si visita prima il sottoalbero sx, quindi la radice, infine il sottoalbero dx

```
InOrder(T)
1  InOrder(T.left)
2  Print(T.key)
3  InOrder(T.right)
4  return
```



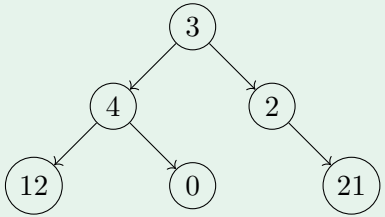
- InOrder dell'esempio stampa: $12, 4, 0, 3, 2, 21$
- Complessità: $\Theta(n)$, tocca una sola volta ogni nodo

Visita di un albero

Visita *anticipata* (*preorder*)

- Nella visita in ordine si visita prima la radice, quindi il sottoalbero sx, infine il sottoalbero dx

```
PreOrder(T)
1  Print(T.key)
2  PreOrder(T.left)
3  PreOrder(T.right)
4  return
```



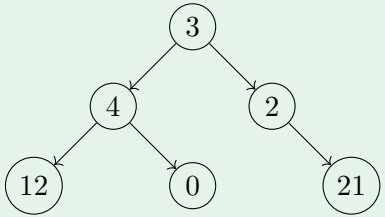
- PreOrder dell'esempio stampa: $3, 4, 12, 0, 2, 21$
- Complessità: $\Theta(n)$, tocca una sola volta ogni nodo

Visita di un albero

Visita *posticipata* (*postorder*)

- Nella visita in ordine si visita prima il sottoalbero sx, poi il sottoalbero dx e infine la radice

PostOrder(T)
1 PostOrder($T.left$)
2 PostOrder($T.right$)
3 Print($T.key$)
4 **return**



- PostOrder dell'esempio stampa: $12, 0, 4, 21, 2, 3$
- Complessità: $\Theta(n)$, tocca una sola volta ogni nodo

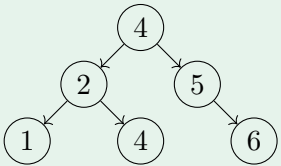
Alberi binari di ricerca (BST)

Definizione

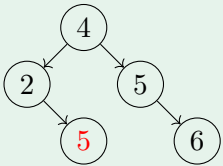
- Uno degli usi più comuni degli alberi binari è utilizzare quelli per cui è valida una data relazione tra le chiavi
- Un albero binario è un *albero binario di ricerca* se per ogni suo nodo x valgono:
  - Se y è contenuto nel sottoalbero sinistro di x, $y.key \leq x.key$
  - Se y è contenuto nel sottoalbero destro di x, $y.key \geq x.key$
- Inserimenti e cancellazioni devono preservare la proprietà
- Una visita in-ordine del BST stampa le chiavi in ordine

Alberi binari di ricerca (BST)

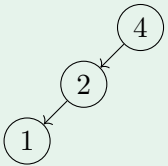
Esempi



É un BST



NON É un BST



É un BST

Nota

- Cambiare la condizione sui valori di chiave rimuovendo la possibilità che siano uguali rende gli elementi del BST unici

Operazioni su BST

Ricerca

- La struttura dei BST li rende naturali candidati per una ricerca efficace degli elementi per chiave

```
RICERCA(T, x)
1  if T = NIL or T.key = x.key
2        return T
3  if T.key < x.key
4        return RICERCA(T.right, x)
5  else return RICERCA(T.left, x)
6
```

- Complessità: $\mathcal{O}(h)$ con h l'altezza dell'albero
- Nel caso ottimo (albero "ben bilanciato") diventa $\mathcal{O}(\log(n))$
- Nel caso pessimo (albero degenere in lista) è $\mathcal{O}(n)$

Operazioni su BST

Minimo e massimo

- L'elemento con chiave minima (massima) è quello più a sinistra (destra) del BST

MIN(T)

```
1  cur ← T
2  while cur.left ≠ NIL
3        cur ← cur.left
4  return cur
```

MAX(T)

```
1  cur ← T
2  while cur.right ≠ NIL
3        cur ← cur.right
4  return cur
```

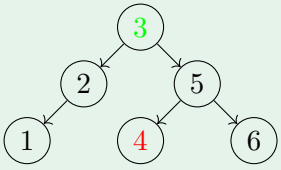
- Complessità: $\mathcal{O}(h)$ con h l'altezza dell'albero

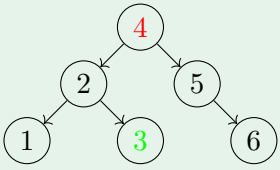
Operazioni su BST

Successore

- Il successore di un elemento x è l'elemento y con la più piccola chiave $y.key > x.key$ presente nel BST
- Nel cercarlo sono possibili due casi:
  1. Il sottoalbero dx di x non è vuoto: il successore è il minimo di quel sottoalbero
  2. Il sottoalbero dx di x è vuoto: il successore è il progenitore più prossimo a x per cui x appare nel suo sottoalbero sx

Esempi: Successore di 3





Operazioni su BST

Successore

- Lo pseudocodice per la ricerca del successore è il seguente:

```
SUCCESSORE(x)
1  if x.right ≠ NIL
2        return MIN(x.right)
3  y ← x.p
4  while y ≠ NIL and y.right = x
5        x ← y
6        y ← y.p
7  return y
```

- Complessità nel caso 1: la stessa del calcolo del minimo: $\mathcal{O}(h)$
- Complessità nel caso 2: caso pessimo, x è la foglia più distante dalla radice, $\mathcal{O}(h)$

Operazioni su BST

Inserimento – Struttura

- L'inserimento di un nuovo elemento deve rispettare la proprietà fondamentale del BST
- Assunzione: non vogliamo che il BST contenga duplicati
- Idea: Cerco l'elemento che voglio inserire nel BST, non lo trovo, lo inserisco al posto del NIL trovato.
- Unica accortezza rispetto al codice della ricerca: tenere traccia dell'ultimo nodo non-NIL per poter inserire correttamente l'elemento

Operazioni su BST

Inserimento – Pseudocodice

```
INSERISCI(T, x)
 1  pre ← NIL
 2  cur ← T.root
 3  while cur ≠ NIL
 4        pre ← cur
 5        if x.key < cur.key
 6              cur ← cur.left
 7        else cur ← cur.right
 8  x.p ← pre
 9  if pre = NIL
10        T.root ← x
11  elseif x.key < pre.key
12        pre.left ← x
13  else pre.right ← x
```

- Le righe 3–7 effettuano la ricerca della posizione di inserimento nell'albero
- Le righe 8–13 effettuano l'inserimento vero e proprio
- Complessità: la stessa della ricerca $\mathcal{O}(h)$ più una porzione a tempo costante (inserimento)

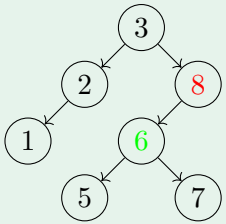
Operazioni su BST

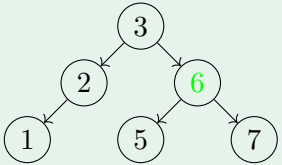
Cancellazione – Struttura – 1

- La strategia di cancellazione di un elemento da un BST dipende dal numero di figli dell'elemento in questione

Caso 1 L'elemento non ha figli: è sufficiente eliminarlo dall'albero deallocandolo e impostando il puntatore del padre a NIL

Caso 2 L'elemento ha un figlio: L'elemento viene sostituito dal figlio nel suo ruolo nell'albero

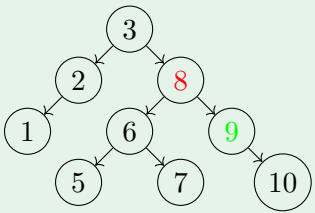




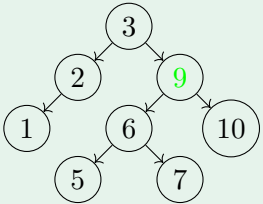
Operazioni su BST

Cancellazione – Struttura – 2

- Caso 3 L'elemento ha due figli: copio il valore del suo successore su di esso ed elimino il successore
- Il successore s di un elemento con due figli x non ha mai il figlio sx f: si avrebbe $x.key < f.key < s.key$, ma questo è impossibile per definizione di successore



Prima



Dopo

Operazioni su BST

Cancellazione – Pseudocodice

```
CANCELLA(T, x)
 1  if x.left = NIL or x.right = NIL
 2        da_canc ← x
 3  else da_canc ← SUCCESSORE(x)
 4  if da_canc.left ≠ NIL
 5        sottoa ← da_canc.left
 6  else sottoa ← da_canc.right
 7  if sottoa ≠ NIL
 8        sottoa.p ← da_canc.p
 9  if da_canc.p = NIL
10        T.root ← sottoa
11  elseif da_canc = da_canc.p.left
12        da_canc.p.left ← sottoa
13  else da_canc.p.right ← sottoa
14  if da_canc ≠ x
15        x.key ← da_canc.key
16  FREE(da_canc)
```

- Le righe 1–3 individuano il nodo da cancellare
- Le righe 4–8 individuano il sottoalbero da spostare e correggono il riferimento al padre
- Le righe 9–13 correggono il riferimento del padre
- Le righe 14–15 copiano il valore della chiave

Analisi di complessità

Sommario

- Tutte le operazioni sono $\mathcal{O}(h)$ con h l'altezza del BST
- Nel migliore dei casi $h = \log(n)$ (albero completo o quasi completo), nel peggiore $h = n$ (lista)
- É critico per avere buone prestazioni mantenere il BST il più possibile vicino al caso ottimo
- Si può dimostrare che l'altezza attesa di un BST è $\mathcal{O}(\log(n))$ se le chiavi inserite hanno distribuzione uniforme
- Volendo un metodo deterministico ci serve una definizione di albero *ben bilanciato*

Albero bilanciato

Vicinanza all'albero completo

- Intuitivamente, vogliamo che la distanza delle foglie dalla radice sia limitata superiormente, per tutte le foglie
- Una definizione operativa (Adelson-Velskii e Landis, 1962)
  - Un albero è bilanciato se, per ogni nodo, le altezze dei due sottoalberi differiscono al più di 1
- Adelson-Velskii e Landis proposero, insieme alla definizione, una modifica ai BST ed ai metodi per accedervi in grado di tenerli bilanciati (alberi AVL)
- Vediamo un'ottimizzazione degli alberi AVL che sacrifica parte del bilanciamento per ottenere inserimenti/cancellazioni più efficienti: gli alberi rosso-neri (red-black trees, RB-trees)
  - Sia la perdita in bilanciamento, che il guadagno sono costanti

Alberi rosso-neri

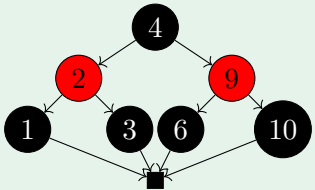
Struttura e definizione

- Un albero rosso-nero è un BST i cui nodi sono dotati di un attributo aggiuntivo, detto *colore* $\in \{rosso, nero\}$, e soddisfacente le seguenti 5 proprietà:
  1. Ogni nodo è rosso o nero
  2. La radice è nera
  3. Le foglie sono nere
  4. I figli di un nodo rosso sono entrambi neri
  5. Per ogni nodo dell'albero, tutti i cammini dai suoi discendenti alle foglie contenute nei suoi sottoalberi hanno lo stesso numero di nodi neri
- Chiamiamo, per comodità, altezza nera (black height) di un nodo x il valore $bh(x)$ pari al numero di nodi neri, escluso x se è il caso, nel percorso che va da x alle foglie

Alberi rosso-neri

Convenzioni

- I dati sono mantenuti unicamente nei nodi interni, le foglie sono tutte NIL
- Per semplicità, tutte le foglie sono fisicamente rappresentate da un singolo nodo, il cui unico riferimento `T.nil` è conservato nella struttura dati
- Il padre del nodo radice punta anch'esso a `T.nil`



Alberi rosso-neri

Azioni sugli alberi rosso-neri

- Tutte le operazioni che non vanno a modificare la struttura dell'albero sono identiche ai BST: RICERCA, MIN, MAX, SUCCESSORE, PREDECESSORE
- Le operazioni di INSERISCI e CANCELLA hanno necessità di mantenere le proprietà degli alberi rosso-neri
  - Idea di massima: opero come se si trattasse di un BST generico, dopodichè compenso le eventuali violazioni
- É necessario essere in grado di ri-bilanciare l'albero con modifiche solamente locali (no ricostruzione dell'albero)

Alberi rosso-neri

Teorema (Proprietà di buon bilanciamento)

Un albero RB con n nodi interni ha altezza massima $2\log(n+1)$

Dimostrazione - 1

- Dim. che un sottoalbero con radice x ha almeno $2^{bh(x)} - 1$ nodi interni, per induzione sull'altezza del sottoalbero
  - Caso base (altezza 0): x è una foglia, il sottoalbero contiene almeno $2^{bh(x)} - 1 = 2^0 - 1 = 0$ nodi interni
  - Passo: dato x, entrambi i suoi figli hanno altezza nera $bh(x)$ o $bh(x) - 1$. Dato che l'altezza dei figli è minore di quella di x (per hp. ind.) i loro sottoalberi hanno almeno $2^{bh(x)-1} - 1$ nodi interni. L'albero radicato in x contiene quindi almeno $2^{bh(x)-1} - 1 + 2^{bh(x)-1} - 1 + 2 = 2^{bh(x)}$ nodi

Alberi rosso-neri

Dimostrazione.

- Per la proprietà 4, almeno metà dei nodi su un qualunque percorso radice-foglia sono neri
- L'altezza nera della radice è almeno $\frac{h}{2}$; per quanto detto prima il sottoalbero radicato in essa contiene almeno $2^{\frac{h}{2}} - 1$ nodi
- $n \geq 2^{\frac{h}{2}} - 1$, risolvendo per $h \rightarrow h \leq 2\log(n+1)$ ■

Conseguenze

- Le operazioni che restano invariate tra RB-trees e BST (RICERCA, MAX, MIN, SUCCESSORE) sono $\mathcal{O}(2\log(n+1))$
- Se riesco a riparare le violazioni in maniera efficiente ho anche INSERISCI e CANCELLA) in $\mathcal{O}(2\log(n+1))$

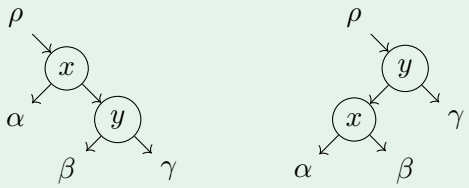
Alberi rosso-neri

Rotazione

Operazione locale a due nodi di un BST che cambia il livello a cui sono situati due nodi senza violare la proprietà BST

$\textsc{LeftRotate}(\mathrm{T},\mathrm{x}) \rightarrow$

$l(\rho) = i - 1$
$l(x) = i$
$l(\alpha) = i + 1$
$l(y) = i + 1$
$l(\beta) = i + 2$
$l(\gamma) = i + 2$



$l(\rho) = i - 1$
$l(x) = i + 1$
$l(\alpha) = i + 2$
$l(y) = i$
$l(\beta) = i + 2$
$l(\gamma) = i + 1$

$\leftarrow \textsc{RightRotate}(\mathrm{T},\mathrm{y})$

Rotazioni

Pseudocodice

LEFTROTATE(T, x)

```
1  y ← x.right // Calcola y
2  x.right ← y.left // Sposta il sottoalbero β
3  if y.left ≠ T.nil // Sistema il riferimento a p della radice di β
4        y.left.p ← x
5  y.p ← x.p // Sistema il riferimento al padre del nodo y
6  if x.p = T.nil // Sistema il riferimento al figlio del padre di y
7        T.root ← y
8  elseif x = x.p.left
9        x.p.left ← y
10 else x.p.right ← y
11 y.left ← x // Aggancia x a sinistra di y
12 x.p ← y
```

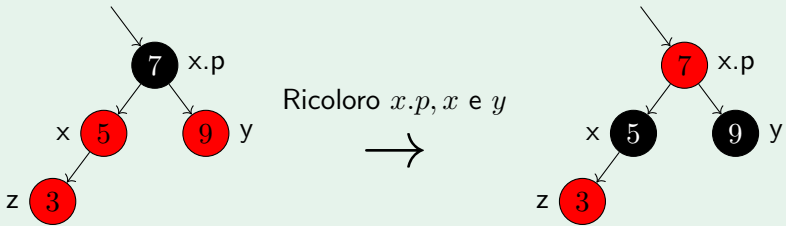
Alberi rosso-neri

Inserimento

- L'inserimento procede ad inserire il nuovo elemento come se l'albero fosse un semplice BST salvo:
  - Assegnare il valore dei sottoalberi del nodo a $T.nil$ al posto di NIL se viene inserito come una foglia
  - Assegnare il valore del genitore del nodo a $T.nil$ al posto di NIL se il nodo è inserito come radice
  - Colorare il nodo appena inserito di rosso
- Possono essere violate la proprietà 4 (i figli di un nodo rosso sono entrambi neri) e la 2 (la radice è nera)
- RiparaRBInserisci(z), dato il nodo inserito z:
  - Con $z.p$ figlio sx del nonno di z: 3 casi a seconda del colore dello "zio" e della posizione di z rispetto a $z.p$
  - Simmetrica per il caso in cui il padre è figlio dx del nonno

RiparaRBInserisci(z)

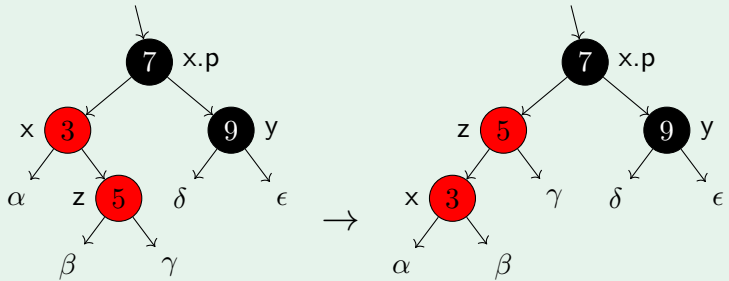
Caso 1 - lo "zio" y è rosso, posizione di z irrilevante, $z.key = 3$



- Successivamente chiamo RiparaRBInserisci($x.p$): $x.p$ potrebbe avere un padre rosso
- Se $x.p$ è la radice, posso colorarla di nero senza problemi

RiparaRBInserisci(z)

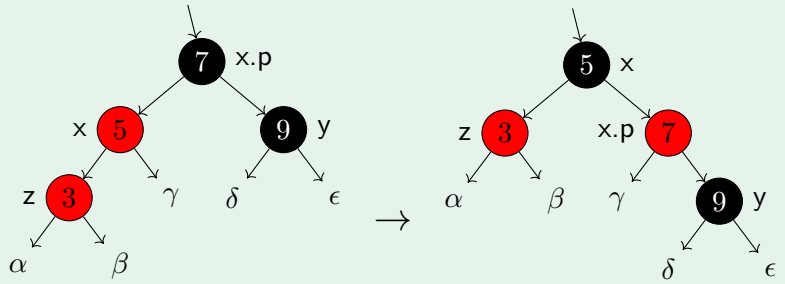
Caso 2 - lo "zio" y è nero, $z = x.right$, $z.key = 5$



- Effettuiamo LeftRotate(T, x): ora la riparazione va effettuata su z, con $z.left = x$ rosso (caso successivo)

RiparaRBInserisci(z)

Caso 3 - lo "zio" y è nero, $z = x.left$, $z.key = 3$



- Scambiamo i colori di x e $x.p$ ed eseguiamo RightRotate($T, p.x$)

Operazioni su alberi RB

RiparaRBInserisci – Pseudocodice

RIPARARBINSERISCI(T, z)

```
 1  while z.p.color = red
 2      if z.p = z.p.p.left
 3          y ← z.p.p.right
 4          if y.color = red // Caso 1
 5              z.p.color ← black
 6              y.color ← black
 7              z.p.p.color ← red
 8              z ← z.p.p
 9          else if z = z.p.right // Caso 2, ruotare z
10                  z ← z.p
11                  LEFTROTATE(z)
12              z.p.color ← black // Caso 3
13              z.p.p.color ← red
14              RIGHTROTATE(z.p.p)
15      else // come le righe 3–14, scambiando right con left
16  T.root.color ← black
```

Alberi rosso-neri - Inserimento

Analisi di complessità

- Nei casi 2 e 3 la procedura $\textsc{RiparaRBInserisci}(z)$ deve solo effettuare un cambio di colori locale e 2 o 1 rotazioni
  - Tutte queste operazioni sono $\mathcal{O}(k)$
- Nel caso 1 continua analizzando il nonno del nodo corrente
  - Caso pessimo: il ciclo itera numero di volte pari a metà dell'altezza dell'intero albero
- L' intera riparazione prende al più $\mathcal{O}(log(n)) \rightarrow$ l'inserimento, comprensivo di riparazione, in alberi RB è $\mathcal{O}(\log(n)$

Cancellazione

- La cancellazione procede a cancellare il nuovo elemento come se l'albero fosse un semplice BST salvo:
  - l' uso di $T.nil$ al posto di NIL
  - invocare la procedura che ripara eventuali violazioni delle proprietà RB
- Nel caso sia eliminato un nodo rosso, non è necessario alcun cambiamento (non sono possibili violazioni delle proprietà)
- Se il nodo eliminato è nero, $\textsc{RiparaRBCancella}(x)$, dato il nodo x presente al posto di quello cancellato z ripristina le proprietà

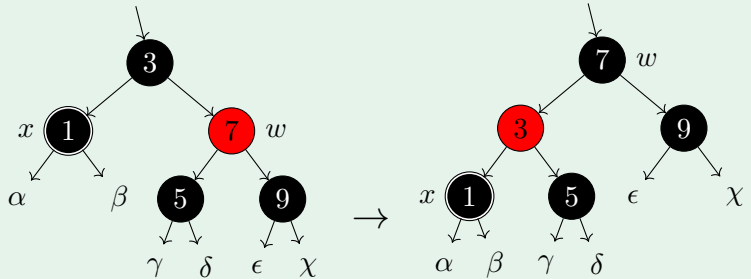
Struttura di RiparaRBCancella(x)

- I 5 casi della procedura RiparaRBCancella(x), con x figlio sx di $x.p$ (procedura simmetrica se figlio dx) sono:
- Caso 0: x è rosso
- Se x è nero
  - Caso 1: Il suo fratello è rosso
  - Se il suo fratello è nero
    - Caso 2: x ha entrambi i nipoti neri
    - Caso 3: x ha entrambi il nipote sinistro rosso
    - Caso 4: x ha entrambi il nipote destro rosso
- La riparazione da effettuare deve sistemare il fatto che x ha un "colore nero aggiuntivo oltre al proprio": il nero che ha "ereditato" dal nodo cancellato

Cancellazione - Caso 0: x è rosso

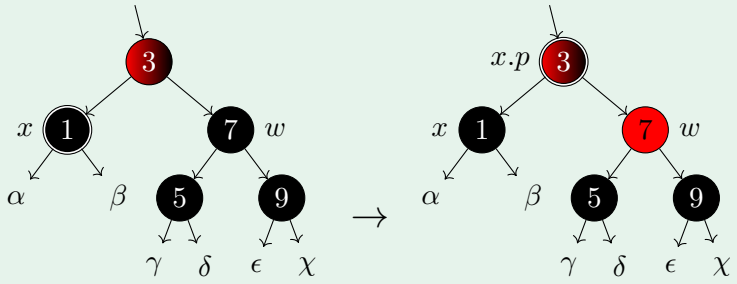
- Viene colorato di nero: ripristina i valori di bh senza violazioni

Cancellazione - Caso 1: x è nero, con fratello w rosso ($\Rightarrow x.p$ nero)



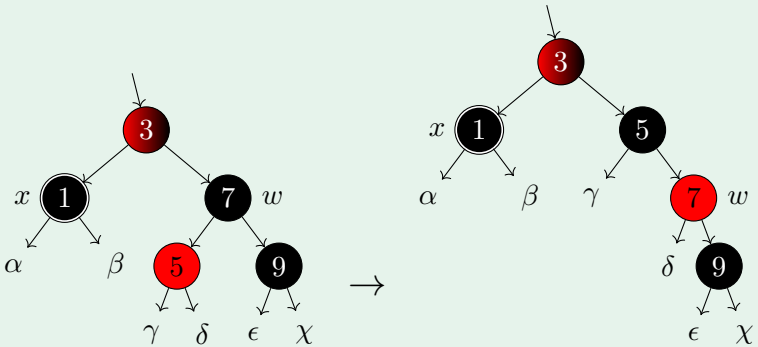
Scambio i colori di w e $w.p$; LeftRotate($x.p$): x ha fratello w nero: $\rightarrow$ casi 2,3,4

Cancellazione - Caso 2: x è nero, con fratello w nero, e nipoti entrambi neri (n.b. $x.p$ può essere nero o rosso)



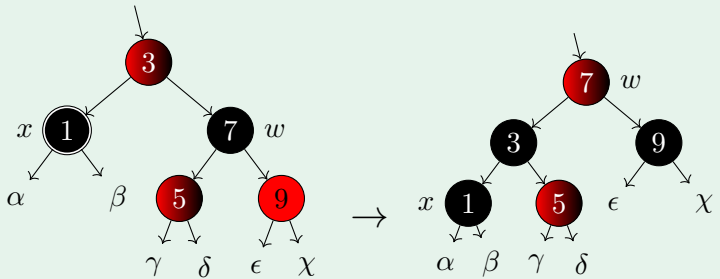
Coloro w di rosso (i.e. rimuovo un nero dal sottoalbero) e richiamo RiparaRBCancella su $x.p$

Cancellazione - Caso 3: x è nero, con fratello w nero, e nipote dx nero (n.b. $x.p$ può essere nero o rosso)



Scambio di colore w e $w.left$, RightRotate su $w \rightarrow$ Caso 4

Cancellazione - Caso 4: x è nero, con fratello w nero, e nipote dx rosso (n.b. $x.p$ e $w.left$ possono essere neri o rossi)



w prende il colore di $w.p$, $w.right$ diventa nero. Invoco LeftRotate su $w.p$

Operazioni su alberi RB

RiparaRBCancella – Analisi di complessità

- I casi 0,1,3 e 4 di RiparaRBCancella effettuano un numero costante di rotazioni e scambi di colore $\rightarrow$ sono $\mathcal{O}(k)$
- L'unica chiamata ricorsiva è quella che avviene nel caso 2 sul padre di x
  - Nel caso il padre ricada nei casi 0,1,3 o 4, la nuova chiamata termina in $\mathcal{O}(k)$
  - In caso contrario, viene ri-invocata RiparaRBCancella
- Ad ogni chiamata ricorsiva si risale di un livello verso la radice $\rightarrow$ al massimo effettuiamo $\mathcal{O}(\log(n))$ chiamate
- La procedura complessiva di cancellazione da alberi RB è quindi $\mathcal{O}(\log(n))$ come tutte le altre azioni

Strutture dati - Parte 3

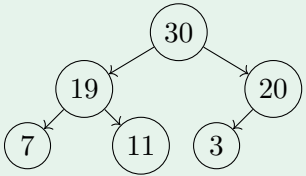
Dipartimento di Elettronica, Informazione e Bioingegneria
Politecnico di Milano

22 maggio 2024

Mucchi (Heaps)

Una struttura parzialmente ordinata

- Un *mucchio* (*heap*) è una struttura dati ad albero la chiave del nodo padre è sempre maggiore (max-heap) di quella dei figli
  - Nessuna relazione sussiste tra le chiavi di due fratelli
  - É possibile definirne una variante in cui la chiave del padre è sempre minore di quella dei figli (min-heap)
- Se l'albero è binario, parliamo di mucchi binari (binary heaps)
  - Manteniamo lo heap come un albero quasi-completo
- Esempio di max-heap:



Mucchi (Heaps)

Proprietà e usi pratici

- Gli heap, in particolare gli heap binari, trovano uso per:
  - Implementare code con priorità
  - Ordinare vettori (proposti originariamente per questo)
- Per tutti gli usi più comuni, è conveniente materializzare lo heap sempre come struttura dati implicita
  - É un albero binario quasi-completo $\rightarrow$ le foglie mancanti sono quelle che occupano la parte finale dell'array in cui è stoccato
  - Avremo un attributo $A.heapsize$ che indica il numero di elementi dello heap e $A.length$ che contiene la lunghezza dell'array di supporto: $A.heapsize \leq A.length$
- Le operazioni su un max-heap sono: MAX, INSERISCI, CANCELLA-MAX, COSTRUISCI-MAX-HEAP, MAX-HEAPIFY
- In un max-heap l'elemento con chiave più grande è la radice

Mucchi (Heaps)

Code con priorità

- Una coda con priorità è una struttura dati a coda in cui è possibile dare una priorità numerica agli elementi all'interno
- Elementi con priorità maggiore verranno estratti sempre prima di elementi con priorità minore indipendentemente dall'ordine di inserimento
- L'implementazione più comune di una coda con priorità è un max-heap
  - La priorità di un elemento è data dalla sua chiave
- Per implementare le primitive necessarie (MAX, INSERISCI, CANCELLA-MAX) necessitiamo di una procedura di supporto:MAX-HEAPIFY

Mucchi (Heaps)

MAX-HEAPIFY

- MAX-HEAPIFY(A, n) riceve un array e una posizione in esso: assume che i due sottoalberi con radice stoccata in LEFT$(n) = 2n$ e RIGHT$(n) = 2n + 1$ siano dei max-heap [a]
- Modifica A in modo che l'albero radicato in n sia un max-heap
- Consente rendere un array A un max-heap come segue:

COSTRUISCI-MAX-HEAP(A)

1 $A.heapsize \leftarrow A.length$
2 **for** $i \leftarrow \lfloor \frac{A.length}{2} \rfloor$ **downto** 1
3 MAX-HEAPIFY(A, i)

[a] Convenzione: gli indici di A vanno da 1 a $A.length$

Mucchi (Heaps)

Max-Heapify

```
MAX-HEAPIFY(A, n)
 1  l ← LEFT(n)
 2  r ← RIGHT(n)
 3  if l ≤ A.heapsize and A[l] > A[n]
 4        posmax ← l
 5  else posmax ← n
 6  if r ≤ A.heapsize and A[r] > A[posmax]
 7        posmax ← r
 8  if posmax ≠ n
 9        SWAP(A[n], A[posmax])
10        MAX-HEAPIFY(A, posmax)
```

- La procedura causa la discesa del nuovo valore verso le foglie sino al punto in cui è maggiore dei figli
- Complessità: nel caso pessimo $\mathcal{O}(\log(n))$ in un heap contenente n elementi

Code con priorità

MAX(A) (esamina l'elemento a priorità massima)

```
MAX(A)
1  return A[1]
```

- L'ispezione è $\mathcal{O}(1)$

CANCELLA-MAX(A) (estrae l'elemento a massima priorità)

```
CANCELLA-MAX(A)
1  if A.heapsize < 1
2        return ⊥
3  max ← A[1]
4  A[1] ← A[A.heapsize]
5  A.heapsize ← A.heapsize − 1
6  MAX-HEAPIFY(A, 1)
7  return max
```

- Estrarre l'elemento a priorità massima costa $\mathcal{O}(\log(n))$
- Mediamente, il costo è inferiore
- Più efficace di un vettore ordinato $\mathcal{O}(\log(n))$ contro $\mathcal{O}(n)$

Code con priorità

INSERISCI(A, key) (accoda un nuovo elemento)

INSERISCI(A, key)

1 $A.heapsize \leftarrow A.heapsize + 1$
2 $A[A.heapsize] \leftarrow key$
3 $i \leftarrow A.heapsize$
4 **while** $i > 1$ and $A[\text{PARENT}(i)] < A[i]$
5 SWAP$(A[\text{PARENT}(i)], A[i])$
6 $i \leftarrow \text{PARENT}(i)$

- Inserisce l'elemento nuovo come ultima foglia
- Fa scalare l'elemento fin quando non è minore del padre
- Complessità nel caso pessimo: $\mathcal{O}(\log(n))$

Code con priorità

Riassunto delle complessità

- Una coda con priorità implementata con uno heap binario ha un costo, sia per l'accodamento che per l'estrazione, pari a $\mathcal{O}(\log(n))$
- Inserendo gli elementi uno alla volta, si ha un costo complessivo di $\mathcal{O}(n \log(n))$ per la costruzione dell'intera coda
- Apparentemente, il costo è identico a quello della COSTRUISCI-MAX-HEAP
- É in realtà possibile dimostrare che COSTRUISCI-MAX-HEAP risulta essere in grado di costruire lo heap in $\mathcal{O}(n)$

Code con priorità

Un limite più preciso

- Uno heap binario è sempre alto $\lfloor \log(n) \rfloor$, il numero di nodi con distanza dalle foglie di h archi è $\leq \frac{2^{\lfloor \log(n) \rfloor}}{2^h} \leq \frac{n}{2^h}$
- Calcolare MAX-HEAPIFY per un nodo, richiede al più $\mathcal{O}(h)$ spostamenti verso il basso
- Calcoliamo il costo complessivo, sommando, per ogni livello, il costo di mucchificare ogni elemento di esso; otteniamo: $\sum_{h=0}^{\lfloor \log(n) \rfloor} \frac{n}{2^h} \mathcal{O}(h) = n\mathcal{O}(\sum_{h=0}^{\lfloor \log(n) \rfloor} \frac{h}{2^h})$, dove, ricordando che $\sum_{h=0}^{\infty} \frac{h}{2^h}$ converge, abbiamo $\mathcal{O}(n)$
- É noto anche il risultato esatto del numero massimo di confronti: $2n - 2s_2(n) - e_2(n)$ dove $s_2(n)$ è il numero di cifre 1 nella rappresentazione binaria di n e $e_2(n)$ è l'esponente del fattore 2 nella fattorizzazione di n

Ordinamento di un vettore

Ordinare con i mucchi

- Ordinare un array in ordine crescente può essere fatto trovando il massimo tra i suoi elementi e posizionandolo alla fine, quindi ripetendo il procedimento sulla parte disordinata
  - Praticamente, è il SelectionSort
- Tuttavia, SelectionSort è $\mathcal{O}(n^2)$ perchè trovare il massimo nella porzione disordinata costa $\mathcal{O}(n)$
- Cosa succede se rendiamo prima l'array un max-heap?

HeapSort(A)

HeapSort(A)

```
1  Costruisci-Max-Heap(A)
2  for i ← A.length downto 2
3      Swap(A[1], A[i])
4      A.heapsize ← A.heapsize − 1
5      Max-Heapify(A, 1)
```

- Riordino gli elementi di A in un max-heap
- Scambio il più grande con l'ultima foglia
- Decremento la dimensione dello heap e riordino l'elemento messo in testa

HeapSort

Considerazioni

- HeapSort ha complessità $\mathcal{O}(n \log(n))$ nel caso pessimo: è la migliore possibile
- Necessita solamente di $\mathcal{O}(1)$ in spazio ausiliario (ordina sul posto) a differenza di MergeSort
- Nelle implementazioni pratiche, nel caso medio, è più lento di QuickSort: HeapSort ha un costo lineare sommato a $\mathcal{O}(n \log(n))$ che viene sempre pagato
- Resta il vantaggio della complessità di caso pessimo garantita
- Come la versione di MergeSort out-of-place HeapSort non è stabile

Grafi

Una struttura dati molto flessibile

- La struttura dati più naturale per rappresentare un insieme di oggetti legati da una generica relazione tra di loro è il *grafo*
- La relazione tra oggetti è rappresentata da un insieme di coppie di oggetti (ordinate o meno)
- Esempio: Una mappa stradale: nodi $\rightarrow$ città, relazione $\rightarrow$ le città sono collegate da una strada

Definizione (Grafo)

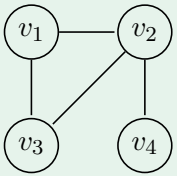
Un grafo è una coppia $\mathcal{G} = (\mathbf{V}, \mathbf{E})$ con $\mathbf{V}$ un insieme di *nodi* (detti anche *vertici*) ed $\mathbf{E}$ un insieme di *archi* (detti anche *lati*).

Grafi

Nomenclatura

- Se un grafo ha $|\mathbf{V}|$ nodi, esso ha al più $|\mathbf{V}|^2$ archi
- Due nodi collegati da un arco si dicono *adiacenti*
- Un *cammino* tra due nodi v_1, v_2 è un insieme di archi di cui il primo ha origine in v_1, l'ultimo termina in v_2 e ogni nodo compare almeno una volta come destinazione di un arco che come sorgente

Esempio



- $\mathbf{V} = \{v_1, v_2, v_3, v_4\}$
- $\mathbf{E} = \{(v_1, v_2), (v_1, v_3), (v_2, v_3), (v_2, v_4)$
  $(v_2, v_1), (v_3, v_1), (v_3, v_2), (v_4, v_2)\}$
- Cammino v_3–v_4: $(v_3, v_2), (v_2, v_4)$

Nomenclatura - 2

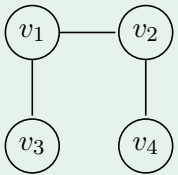
- Un grafo è detto *orientato* (*directed graph*) se la coppia di nodi che costituisce un arco è ordinata.
  - In altre parole, se ciò che collega i nodi ha un "verso"
  - I suoi archi sono detti archi orientati (*arcs*, non *edges*)
  - Esempio: un albero è un grafo orientato
- Un grafo orientato viene rappresentato indicando gli archi come frecce che puntano al secondo nodo della coppia
- In un grafo non orientato, l'insieme degli archi può essere rappresentato in modo compatto dato che se $(v_1, v_2) \in \mathbf{E}$, allora anche $(v_2, v_1) \in \mathbf{E}$

Grafi

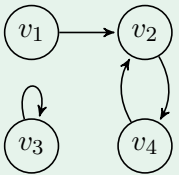
Nomenclatura - 2

- grafo *connesso*: esiste un percorso per coppia di nodi
- grafo *completo* (*completamente connesso*): esiste un arco per ogni coppia di nodi
- Un percorso è un *ciclo* se il nodo di inizio e di fine coincidono
  - Il ciclo è orientato se segue la direzione degli archi
- Un grafo privo di cicli è detto *aciclico*

Esempi



Grafo
non orientato
connesso
aciclico



Grafo
orientato
non connesso
ciclico

Grafi

Rappresentazione in memoria

- Sono possibili due strategie per rappresentare un grafo: liste di adiacenza e matrice di adiacenza
- Liste di adiacenza:
  - Un vettore di liste lungo $|\mathbf{V}|$, indicizzato dai nomi dei nodi
  - Ogni lista contiene i nodi adiacenti all'indice della sua testa
- Matrice di adiacenza:
  - Una matrice di valori booleani $|\mathbf{V}| \times |\mathbf{V}|$, con righe e colonne indicizzate dai nomi dei nodi
  - la cella alla riga i, colonna j contiene 1 se l'arco (v_i, v_j) è presente nel grafo (0 altrimenti)

Grafi

Rappresentazioni a confronto

- Complessità spaziale: Liste: $\Theta(|\mathbf{V}| + |\mathbf{E}|)$, Matrice $\Theta(|\mathbf{V}|^2)$
  - La rappresentazione a liste è più compatta se il grafo è sparso ovvero se il numero di archi è "basso": $|\mathbf{E}| \ll |\mathbf{V}|^2$
- Complessità temporale per determinare:
  - Se (v_1, v_2) appartiene a un grafo: Liste: $\mathcal{O}(|\mathbf{V}|)$, Matrice $\mathcal{O}(1)$
  - Il numero di archi o_e uscenti da un nodo: Lista: $\Theta(o_e)$, Matrice $\mathcal{O}(|\mathbf{V}|)$

Ottimizzazioni per grafi non orientati

- La matrice di adiacenza di un grafo non orientato è simmetrica rispetto alla diagonale principale: posso stoccarne solo metà
- Liste di adiacenza: posso stoccare solo uno dei due archi e raddoppiando il tempo di ricerca per un nodo adiacente

Grafi

Operazioni su grafi

- Le operazioni su grafi sono tipicamente di ispezione:
  - Visita in ampiezza
  - Visita in profondità
- ... o vanno a determinare proprietà del grafo:
  - Trovare le componenti connesse
  - Ordinamento topologico
  - Percorso più breve tra due nodi
  - Individuare cicli

Visita in ampiezza (Breadth First Search, BFS)

- La strategia di visita *in ampiezza* visita tutti i nodi di un grafo $\mathcal{G}$ a partire da uno nodo sorgente s
  - Ordine di visita: vengono visitati tutti i nodi con un cammino tra loro e s lungo n passi, prima di visitare quelli con un cammino lungo $n + 1$
- La visita di un grafo è più problematica di quella di un albero: possono essere presenti cicli
  - Evitiamo di iterare all'infinito colorando i nodi mentre li visitiamo:
    - Nodo bianco: deve essere ancora visitato
    - Nodo grigio: il nodo è stato visitato, devono essere visitati quelli adiacenti ad esso
    - Nodo nero: sono stati visitati sia il nodo che quelli adiacenti

Visita in ampiezza – Schema

- Memorizziamo in una coda (semplice, senza priorità) i nodi ancora da visitare
- La coda è inizializzata con la sola sorgente
- Estraiamo un nodo dalla coda e:
  - Visitiamo i vicini bianchi
  - Li coloriamo di grigio (possiamo calcolare la loro distanza da s)
  - Li accodiamo affinchè siano visitati a loro volta
- Marchiamo quindi il nodo estratto come nero e riprendiamo estraendo il successivo

Visita in ampiezza

Pseudocodice - VisitaAmpiezza(G, s)

VisitaAmpiezza(G, s)

```
1  for each  n ∈ V \ {s}
2        n.color ← white
3        n.dist ← ∞
4  s.color ← grey
5  s.dist ← 0
6  Q ← ∅
7  Enqueue(Q, s)
8  while ¬IsEmpty(Q)
9        curr ← Dequeue(Q)
10       for each  v ∈ curr.adiacenti
11             if v.color = white
12                   v.color ← grey
13                   v.dist ← curr.dist + 1
14                   Enqueue(Q, v)
15       curr.color ← black
```

- Linee 1–7 : inizializzano tutti i nodi come bianchi
- Linee 8–15: effettuano la visita del grafo
- N.B. Ogni arco è visitato una sola volta (a partire dal nodo sorgente)
- Complessità totale: $\mathcal{O}(|\mathbf{V}| + |\mathbf{E}|)$

Grafi

Utilizzi

- Come per le visite degli alberi è possibile stampare i nodi in una visita di un grafo
- VisitaAmpiezza si trasforma in algoritmo di ricerca
  - Basta inserire un controllo appena si sta per accodare un nuovo elemento: se è quello corretto lo si ritorna

Visita in profondità

- Diversamente dalla visita in ampiezza, visitiamo prima i nodi adiacenti a quello dato, poi il nodo stesso
  - Segue i cammini "fino in fondo" prima di visitare i vicini del nodo di partenza
  - Il codice è identico a VisitaAmpiezza sostituendo la coda con una pila (condivide quindi anche le complessità)

Grafi

Componenti connesse

- É detta *componente connessa* di un grafo $\mathcal{G}$ un insieme $\mathbf{S}$ di nodi tali per cui esiste un cammino tra ogni coppia di essi, ma nessuno di essi è connesso a nodi $\notin \mathbf{S}$
- Individuare le componenti connesse in un grafo equivale ad etichettare i nodi con lo stesso valore se appartengono alla stessa componente
- Molto utile nella pratica:
  - Individua punti non serviti nella mappa di una città se rappresentata con un grafo
  - Se il grafo rappresenta la relazione di amicizia di un social network equivale a individuare le comunità

Componenti connesse

ComponentiConnesse(G) (etichette intere per le comp.)

ComponentiConnesse(G)

```
1  for each v ∈ V
2        v.etichetta ← −1
3  eti ← 1
4  for each v ∈ V
5        if v.etichetta = −1
6              VisitaEdEtichetta(G, v, eti)
7              eti ← eti + 1
```

- VisitaEdEtichetta funziona come VisitaAmpiezza o VisitaProfondità ma imposta a eti il campo $etichetta$ del nodo visitato
- Complessità: $\mathcal{O}(|\mathbf{V}| + |\mathbf{E}|)$, ogni nodo viene visitato una sola volta

Ordinare i nodi di un grafo

Definizione (Predecessore)

Dato un grafo orientato, il *predecessore* di un nodo v è un nodo u tale per cui esiste un cammino da u a v

- Percorrendo il grafo lungo gli archi, a partire da u *possiamo* raggiungere v
  - non lo raggiungiamo necessariamente, possono esserci biforcazioni

Ordinamento Topologico

- Un valore utile da calcolare per un grafo *orientato aciclico* è il cosidetto ordinamento topologico
- L'ordinamento topologico è una sequenza di nodi del grafo tale per cui nessun nodo compare prima di un suo predecessore
  - N.B. L'ordinamento topologico non è unico!

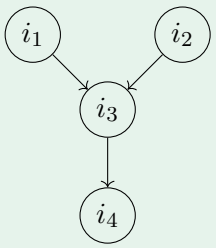
Esempio

Grafo delle dipendenze dati (DDG) $(\mathbf{V}, \mathbf{E})$

- $\mathbf{V}$: istruzioni $i_j, j \in \mathbb{N}$; $\mathbf{E}$: $(i_j, i_k) \in \mathbf{E}$ sse i_k usa il risultato prodotto da i_j

PITAGORA(a, b)

1 $aq \leftarrow a^2$
2 $bq \leftarrow b^2$
3 $cq \leftarrow aq + bq$
4 **return** $\sqrt{cq}$



- Ordinamenti topologici validi: i_1, i_2, i_3, i_4 e i_2, i_1, i_3, i_4
- Sono ordini in cui si può eseguire le istruzioni, certi di avere già calcolato gli operandi

Calcolare l'ordinamento topologico

Osservazione

- Se un grafo non è connesso, le componenti connesse possono essere ordinate in qualunque modo l'una rispetto all'altra

Idea della procedura

- Per calcolare un ordinamento topologico è possibile riusare la procedura di VisitaProfondità
- Quando coloriamo un nodo di nero lo inseriamo in testa ad una lista

Ordinamento Topologico

OrdinamentoTopologico(G)

```
1  for each v ∈ V
2        v.color ← white
3  for each v ∈ V
4        if v.color = white
5              VisitaProfOT(G, v, L)
6  return L
```

VisitaProfOT(G, s, L)

```
1  s.color ← grey
2  for each  v ∈ curr.adiacenti
3        if v.color = white
4              VisitaProfOT(G, v, L)
5  s.color ← black
6  PushFront(L, s)
```

Il percorso più breve

Rivisitando l'algoritmo di Dijkstra

- Trova, dato un grafo orientato e un suo nodo s, i percorsi più brevi da un nodo a qualunque altro
  - Funziona sia su di un grafo classico, che su di un grafo *pesato* ovvero con archi dotati di un valore intero
- Principio di funzionamento
  - Inserisco ogni $v \in \mathbf{V} \setminus \{s\}$ in un insieme $\mathbf{Q}$ dopo aver impostato il suo attributo distanza a ∞ ed il $v.pred$ a NIL
  - Inserisco s in $\mathbf{Q}$ dopo aver impostato $s.dist \leftarrow 0$, $s.pred \leftarrow NIL$
  - Fin quando $\mathbf{Q}$ non è vuoto, estraggo il nodo c con $dist$ minima e controllo per ogni adiacente a se hanno distanza minore di $c.dist + peso(c, a)$
    - Se questo accade imposto $a.pred \leftarrow c, a.dist \leftarrow c.dist + peso(c, a)$

Rivisitando l'algoritmo di Dijkstra

- La proposta originale di Dijkstra stocca l'insieme $\mathbf{Q}$ come un vettore
  - L'algoritmo effettua nel caso pessimo (grafo completamente connesso) $\mathcal{O}(|\mathbf{V}|)$ accessi ad ogni controllo per le distanze
  - Viene effettuato un controllo per ogni nodo del grafo $\rightarrow$ Comp. temporale $\mathcal{O}(|\mathbf{V}|^2)$
- Possiamo migliorare la complessità memorizzando l'insieme $\mathbf{Q}$ come una coda (min-heap) con priorità (la distanza)
  - Ogni estrazione di nodo a priorità minima costa $\mathcal{O}(1)$
  - Ogni cambio di priorità $\mathcal{O}(\log(|\mathbf{V}|)$

Dijkstra Ottimizzato

```
1  Q ← ∅
2  s.dist ← 0
3  for each v ∈ V
4      if v ≠ s
5          v.dist ← ∞
6          v.pred ← NIL
7      AccodaPri(Q, v, v.dist)
8  while Q ≠ ∅
9      u ← CancellaMin(Q)
10     for each v ∈ u.succ
11         ndis ← u.dist + peso(u, v)
12         if v.dist > ndis
13             v.dist ← ndis
14             v.prev ← u
15             DecrementaPri(Q, v, ndis)
```

- Le righe 3–7 inizializzano la coda: costo $\mathcal{O}(|\mathbf{V}|\log(|\mathbf{V}|)$
- Le righe 8–15 visitano ogni arco una volta (grafo come lista di adiacenze): Costo $\mathcal{O}(|\mathbf{E}|\log(|\mathbf{V}|))$
- Compl. totale $\mathcal{O}((|\mathbf{E}|+|\mathbf{V}|)\log(|\mathbf{V}|))$

Individuare cicli

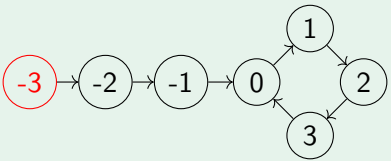
Un problema ricorrente

Dato un grafo orientato, per cui ogni nodo ha un solo successore determinare, dato un nodo di partenza, se il cammino che parte da esso ha cicli

- Utile anche nel caso in cui i successori siano molteplici ma ci sia una regola per sceglierne uno
  - Esempio: Il calcolo fatto da un FSA sta ciclando su un insieme finito di stati?
- Altrettanto utile se la relazione è una funzione matematica: il grafo può non essere materializzato in memoria
  - Esempio: Il calcolo fatto da una MT/programma sta ciclando su un insieme *finito* di configurazioni al posto di terminare?
  - Esempio 2: Una successione matematica dove $x_i = f(x_{i-1})$ si ripete periodicamente? (utile test per generatori di numeri casuali)

Algoritmo di Floyd

La lepre e la tartaruga - Idea



- Immaginiamo che il cammino su cui vogliamo individuare il ciclo sia una pista
- Usiamo due riferimenti t e l che spostiamo a ogni passo:
  - Nel caso di t, dal nodo a cui punta al successore (1 "passo")
  - Nel caso di l, dal nodo a cui punta al successore del successore (2 "passi")
- Entrambi partono dal nodo iniziale (-3 in figura)
- Se esiste un ciclo, essi sono destinati a "incontrarsi" (l doppierà prima o poi t)

Algoritmo di Floyd

La lepre e la tartaruga - Osservazioni e obiettivi

- Chiamiamo C la lunghezza del ciclo (4 nell'esempio) e T quella della "coda" che lo precede (3 nell'esempio)
- Quando t ha effettuato T passi, l si trova sicuramente nella porzione ciclica del grafo (l percorre strettamente più passi)
- Ad ogni mossa successiva l guadagna su t una posizione: la raggiunge sicuramente!
  - Sfruttiamo questo fatto per riconoscere l'esistenza di un ciclo
- Con un po'di analisi siamo in grado di ricavare anche quanto valgono T e C

Algoritmo di Floyd

La lepre e la tartaruga - Riconoscere l'esistenza del ciclo

- N.B. due posizioni $> T$ a distanza di C passi sono la stessa posizione
- Scriviamo per comodità T come $T = qC + r$
- Dopo T mosse l ha quindi effettuato $2T$ passi, di cui T nel ciclo
  - Osserviamo che la sua posizione è quindi $T = qC + r \equiv_C r$ nel ciclo
- Dopo altre $C - r$ mosse, t si trova $C - r$ posizioni all'interno del ciclo, l si trova in $r + 2(C - r) = 2C - r \equiv_C C - r$: si sono incontrate
- Il numero di mosse totali prima dell'incontro è quindi $T + C - r = (qC + r) + C - r = (q + 1)C$: si incontrano dopo un numero di mosse multiplo della lunghezza del ciclo

Algoritmo di Floyd

La lepre e la tartaruga - Calcolare T e C

- l e t hanno compiuto $(q+1)C$ passi
- Obiettivo: trovare T
  - Riporto t all'inizio, faccio muovere sia l che t, un passo alla volta contando i passi
  - dopo $T = qC + r$ passi, t si trova all'inizio del ciclo, l si trova a $(q+1)C + qC + r \equiv_C qC + r$: si incontrano!
- Per trovare C mi basta tenere ferma t e far procedere l di un passo alla volta, contando i passi, fin quando non si reincontrano

Riconoscimento di cicli

FloydLT(G, x)

FloydLT(G, x)

```
1   t ← x.succ
2   l ← x.succ.succ
3   while l ≠ t
4       t ← t.succ
5       l ← l.succ.succ
6   T ← 0
7   t ← x
8   while l ≠ t
9       t ← t.succ
10      l ← l.succ
11      T ← T + 1
12  l ← t
13  C ← 0
14  while l ≠ t
15      l ← l.succ
16      C ← C + 1
17  return T, C
```

- Le righe 1–5 trovano il ciclo quando l riprende t
- Le righe 6–11 calcolano T facendo ripartire t
- Le righe 14-16 calcolano C tenendo t ferma come segnaposto per l
- Complessità temporale: $\Theta(T + C - r + T + C) = \Theta(2(T + C) - r)$
- Complessità spaziale: $\Theta(1)$